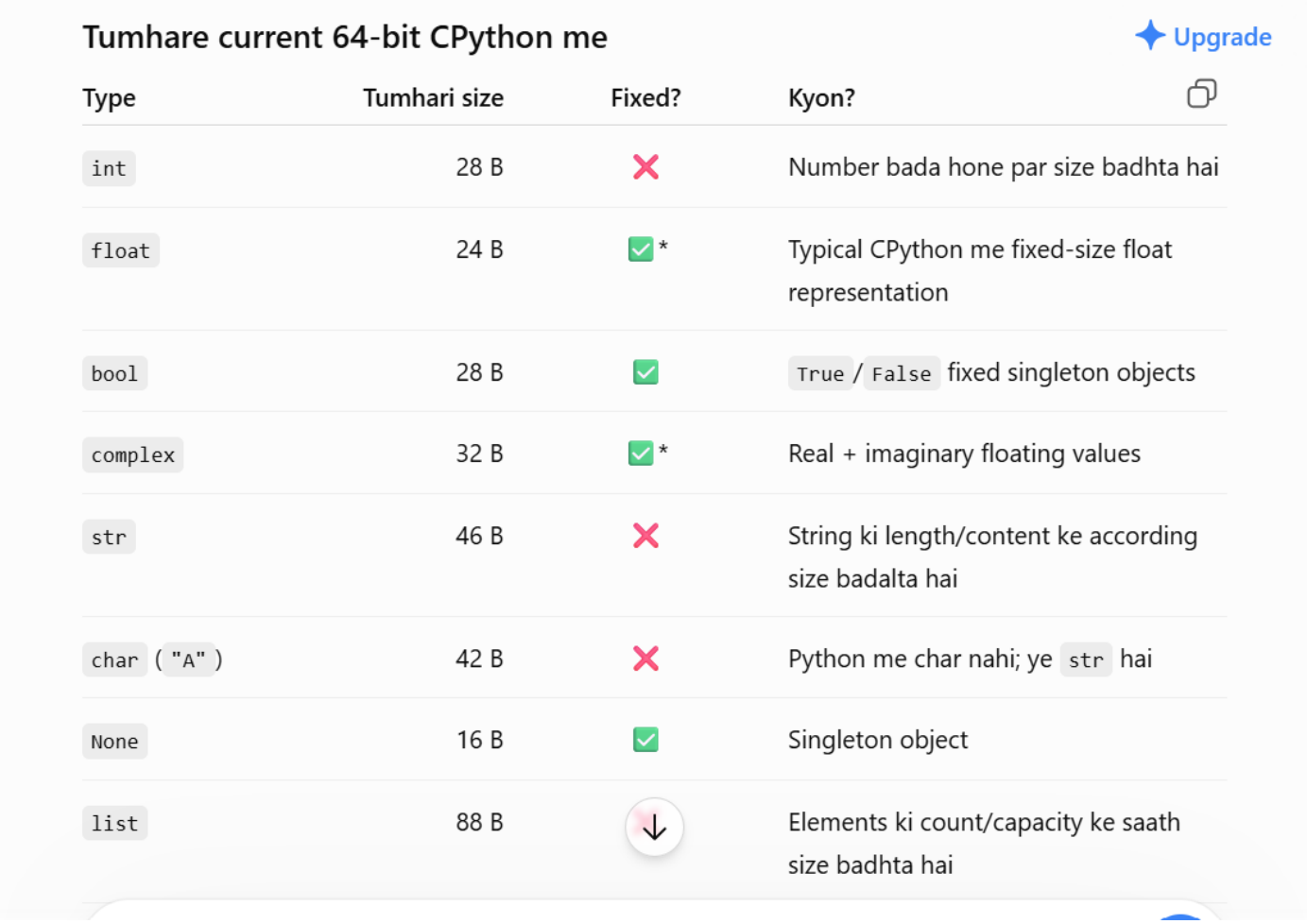
python



Tumhare current 64-bit CPython me

Upgrade

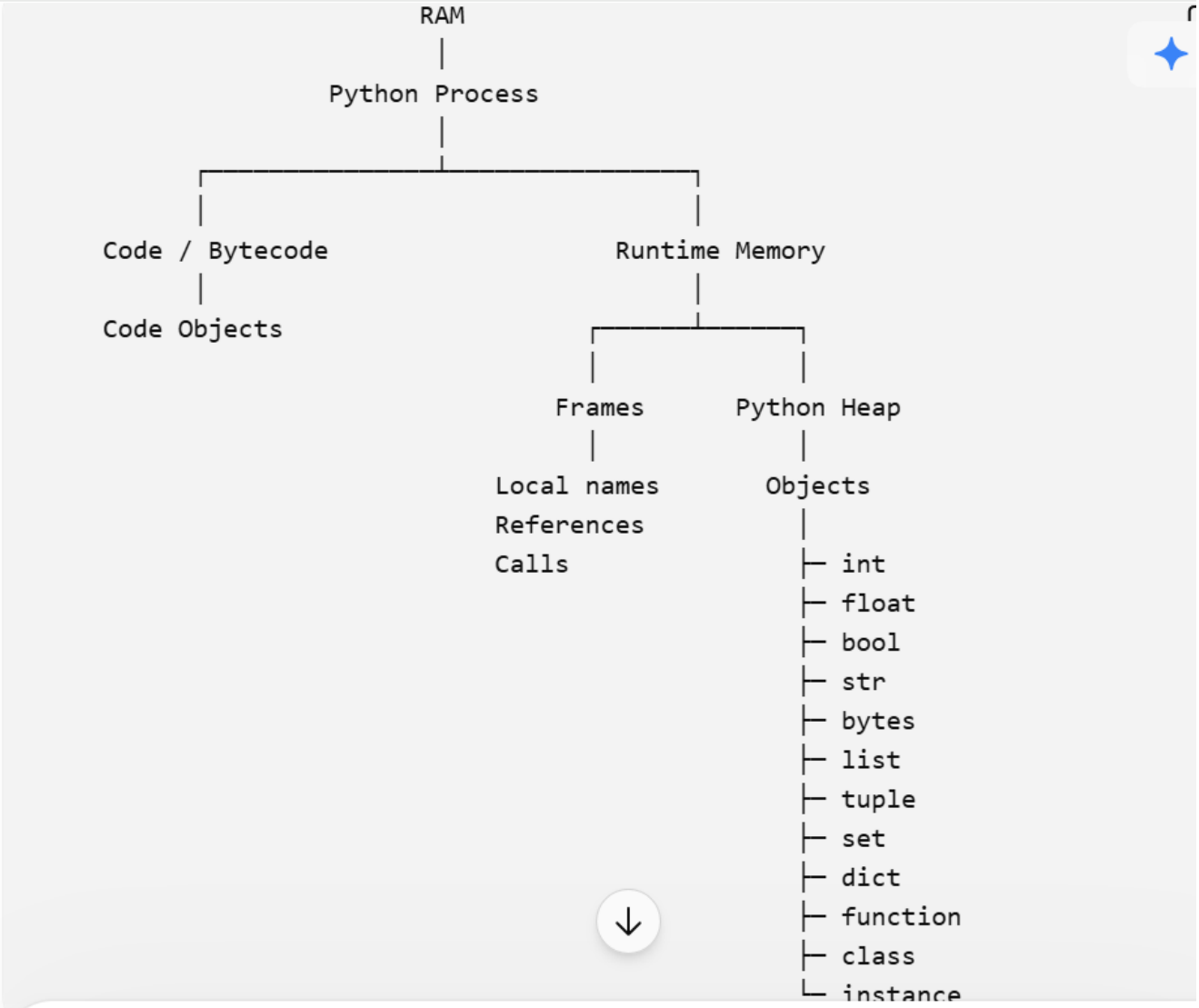
Type	Tumhari size	Fixed?	Kyon?
`int`	28 B	❌	Number bada hone par size badhta hai
`float`	24 B	✅*	Typical CPython me fixed-size float representation
`bool`	28 B	✅	`True` / `False` fixed singleton objects
`complex`	32 B	✅*	Real + imaginary floating values
`str`	46 B	❌	String ki length/content ke according size badalta hai
`char` (`"A"`)	42 B	❌	Python me char nahi; ye `str` hai
`None`	16 B	✅	Singleton object
`list`	88 B	↓	Elements ki count/capacity ke saath size badhta hai

ggg

`list`	88 B	❌	Elements ki count/capacity ke size badhta hai
`tuple`	64 B	❌	Elements ki count ke according size badhta hai
`set`	216 B	❌	Hash-table capacity ke according size badhta hai
`dict`	184 B	❌	Entries/capacity ke according size badhta hai
`bytes`	38 B	❌	Bytes ki length ke according size badhta hai
`bytearray`	62 B	❌	Data/capacity ke according size badhta hai
`date`	32 B	✅*	Typical CPython implementation me fixed object size
`time`	40 B	✅*	Typical CPython implementation me

`time`	40 B	✅*	Typical CPython implementati fixed object size
`datetime`	48 B	✅*	Typical CPython implementation me fixed object size
`array.array`	92 B	❌	Elements ki count ke according badhta hai
`range`	48 B	✅*	Range object ka representation compact/fixed-size hota hai
`frozenset`	216 B	❌	Elements/capacity ke according change hota hai
`type`	416 B	❌	Type objects implementation-dependent hain

python



Python Memory Management kya hai?

Upgrade

> **Python Memory Management ek process hai jisme Python automatically program ke objects ke liye memory allocate karta hai, unhe manage karta hai, aur jab objects ki zarurat nahi hoti to memory ko reclaim/release karta hai.**

Simple Hinglish mein:

Tumhe normally manually memory allocate/free nahi karni padti; Python ka memory management system ye kaam automatically handle karta hai.

Python Memory Management ka flow

```
Python Program
      ↓
Object create hota hai
      ↓
Python memory allocate karta hai
      ↓
Object use hota hai
      ↓
Object ki zarurat khatam
      ↓
Reference count / Garbage Collector check
      ↓
Memory reclaim
```

Python Memory Management ke main components

Component	Kya karta hai?
Private Heap	Python objects ke liye memory area
Memory Allocator	Objects ke liye memory allocate karta hai
Reference Counting	Check karta hai object ko kitne references point kar rahe hain
Garbage Collector (GC)	Un objects ko clean karta hai jo references ke through reachable nahi hain, especially cyclic references
`del`	Reference ko remove kar sakta hai
`gc` **module**	Garbage collection ko inspect/control karne ke tools deta hai

python

C++ vs Python — Interview Point of View

Upgrade Share

#	Point	C++	Python
1	**Type**	Statically typed language	Dynamically typed language
2	**Execution**	Compiled language; source code machine code mein compile hota hai	Typically interpreted/bytecode-based; CPython source ko bytecode mein compile karke execute karta hai
3	**Variable Declaration**	Usually type declare karna padta hai: `int x = 10;`	Type explicitly declare nahi karna padta: `x = 10`
4	**Memory Management**	Manual control possible: `new/delete`, RAII, smart pointers	Automatic memory management; reference counting + garbage collector in CPython
5	**Pointers**	Explicit pointers available: `int *p`	C++ jaise traditional explicit pointers nahi
6	**Data Structures**	Array mein same-type elements contiguous memory mein store ho sakte hain	List references ka contiguous storage use karti hai, lekin referenced objects necessarily contiguous nahi hote
7	**Speed**	Generally faster, especially CPU-intensive applications mein	Generally slower than C++ due to runtime/object overhead
8	**Use Cases**	Game engines, embedded systems, OS/system software, high-performance applications	AI/ML, automation, scripting, data science, web development, rapid development

Python ko **interpreted language** isliye bola jata hai kyunki Python program ka **bytecode Python Virtual Machine (PVM) ke through execute hota hai**, na ki C++ ki tarah directly machine code executable banakar run hota hai.

Simple flow

```
Python Code (.py)
       ↓
Python Compiler
       ↓
Bytecode (.pyc)
       ↓
Python Virtual Machine (PVM)
       ↓
Execution
```

form nahi hain.

- **Interpreter** = jo program/code ko execute karta hai.
- **Interpreted** = jis language ko interpreter ke through execute kiya jata hai.

Example:

```
Python
  ↓
Python Compiler
  ↓
Bytecode
  ↓
PVM / Interpreter
  ↓
Execution
```

Simple analogy

Teacher = Interpreter
Student = Code
Teaching/explaining process = Interpreting

#	Point	Compiler	Interpreter
1	**Code conversion**	**Pura program pehle** target/machine code mein convert karta hai	Code ko **runtime par step-by-step process/translate** karta hai
2	**Kab conversion?**	**Execution se pehle**	**Execution ke time**
3	**Execution**	`Pura code → Convert → Execute`	`Read/Translate → Execute → Next`
4	**Machine code**	Usually native machine code/executable generate karta hai	Har step par permanent machine-code executable banana zaroori nahi
5	**Speed**	Generally execution fast	Traditional interpretation generally slower
6	**Error**	Compila[illegible]hase mein errors detect hote hain	Runtime execution ke dauran error aa sakta hai

python

Python Operator Precedence — High → Low

Priority	Operators	Associativity
1	`()` `[]` `x[index]` `f()`	Left → Right*
2	`**`	**Right → Left**
3	`+x` `-x` `~x`	Right → Left*
4	`*` `/` `//` `%` `@`	Left → Right
5	`+` `-`	Left → Right
6	`<<` `>>`	Left → Right
7	`&`	Left → Right
8	`^`	Left → Right
9	`|`	Left → Right

python

9d01-8ee16ed71cff

9	`|`	Left → Right
10	`==` `!=` `<` `>` `<=` `>=` `in` `is`	Left → Right
11	`not`	Right → Left
12	`and`	Left → Right
13	`or`	Left → Right
14	`if ... else`	Right → Left
15	`:=`	Right → Left
16	`=` , `+=` , `-=` , `*=` , `/=` , `//=` , `%=` etc.	**Right → Left**
17	`,`	Left → Right

ff

python

l01-8ee16ed71cff

Error / Exception	Kab aata hai?	Example
`ValueError`	Value ka type/context galat ho	`int("abc")`
`TypeError`	Wrong type ke saath operation	`"5" + 2`
`NameError`	Variable/name exist nahi karta	`print(x)`
`IndexError`	List/string ka invalid index	`[1,2][5]`
`KeyError`	Dictionary mein key nahi hai	`d["x"]`
`ZeroDivisionError`	0 se divide	`10 / 0`
`AttributeError`	Object mein attribute/method nahi	`"abc".append()`
`FileNotFoundError`	File nahi mili	`open("abc.txt")`

python

`ImportError`	Import related problem	`from x import y`
`ModuleNotFoundError`	Module installed/available nahi	`import xyz`
`SyntaxError`	Python syntax galat	`if x`
`IndentationError`	Indentation galat	wrong spacing
`UnboundLocalError`	Local variable ko assign se pehle use	function case
`RecursionError`	Recursion bahut deep ho gayi	infinite recursion
`OverflowError`	Number calculation range se bahar	very large float operation
`MemoryError`	Memory insufficient	huge allocation

ff

python

```python
import sys
import datetime
from array import array


print("========== BASIC NUMERIC TYPES ==========")

x_int = 10
print("int      :", type(x_int), sys.getsizeof(x_int), id(x_int))

x_float = 10.5
print("float    :", type(x_float), sys.getsizeof(x_float), id(x_float))

x_bool = True
print("bool     :", type(x_bool), sys.getsizeof(x_bool), id(x_bool))

x_complex = 2 + 3j
print("complex  :", type(x_complex), sys.getsizeof(x_complex), id(x_complex))
```

python

```python
print("\n========== STRING / CHARACTER ==========")

x_str = "hello"
print("str      :", type(x_str), sys.getsizeof(x_str), id(x_str))

x_char = "A"
print("char     :", type(x_char), sys.getsizeof(x_char), id(x_char))

# Python mein separate char datatype nahi hota
# Single character bhi str hi hota hai.


print("\n========== NONE ==========")

x_none = None
print("None     :", type(x_none), sys.getsizeof(x_none), id(x_none))


print("\n========== COLLECTION TYPES ==========")

x_list = [1, 2, 3]
print("list     :", type(x_list), sys.getsizeof(x_list), id(x_list))
```

python

```python
x_tuple = (1, 2, 3)
print("tuple    :", type(x_tuple), sys.getsizeof(x_tuple), id(x_tuple))

x_set = {1, 2, 3}
print("set      :", type(x_set), sys.getsizeof(x_set), id(x_set))

x_dict = {"a": 1, "b": 2}
print("dict     :", type(x_dict), sys.getsizeof(x_dict), id(x_dict))


print("\n========== BYTE TYPES ==========")

x_bytes = b"hello"
print("bytes    :", type(x_bytes), sys.getsizeof(x_bytes), id(x_bytes))

x_bytearray = bytearray(b"hello")
print("bytearray:", type(x_bytearray), sys.getsizeof(x_bytearray), id(x_bytearray))
```

```python
print("\n========== ARRAY ==========")

x_array = array("i", [1, 2, 3])
print("array    :", type(x_array), sys.getsizeof(x_array), id(x_array))


print("\n========== DATE / TIME ==========")

x_date = datetime.date.today()
print("date     :", type(x_date), sys.getsizeof(x_date), id(x_date))

x_time = datetime.datetime.now().time()
print("time     :", type(x_time), sys.getsizeof(x_time), id(x_time))

x_datetime = datetime.datetime.now()
print("datetime :", type(x_datetime), sys.getsizeof(x_datetime), id(x_datetime))
```

1. Python Function kya hota hai?

Function ek reusable block of code hota hai jo kisi specific kaam ko perform karta hai.

Python

Run

```python
def add(a, b):
    return a + b

result = add(10, 20)

print(result)        # Output: 30
```

2. Function ke important parts

Python

```python
def greet(name):        # name = parameter
    message = "Hello " + name
    return message

x = greet("Tarun")      # "Tarun" = argument

print(x)                # Output: Hello Tarun
```

> **Pass by object reference / call by sharing**

Matlab function ko **object ka reference** diya jata hai. Ab object **mutable hai ya immutable**, uske according behavior dikhega.

1. Immutable object → change nahi hoga

Python Run

```python
def change(x):
    x = x + 10
    print(x)                    # Output: 20

a = 10

change(a)

print(a)                        # Output: 10
```

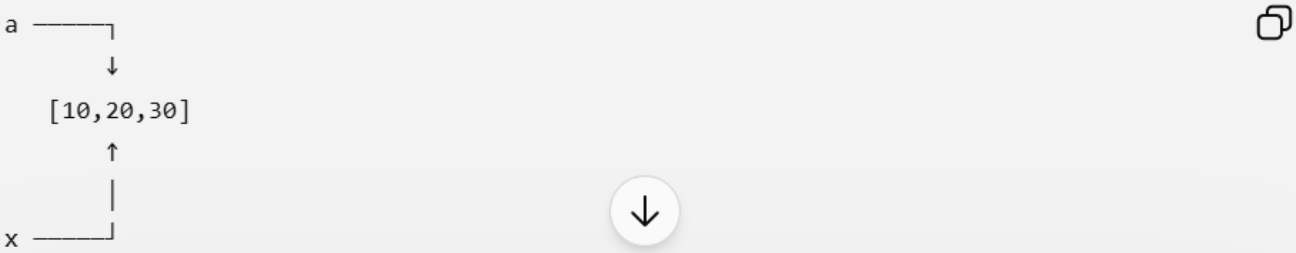
2. Mutable object → original change ho sakta hai

List example:

Python Run

```python
def change(x):
    x.append(40)

a = [10, 20, 30]

change(a)

print(a)                    # Output: [10, 20, 30, 40]
```

Yahan `x` aur `a` **same list object** ko refer kar rahe hain.



3. Lekin mutable object ko reassign karoge to original change nahi होगा

Ye bahut important hai:

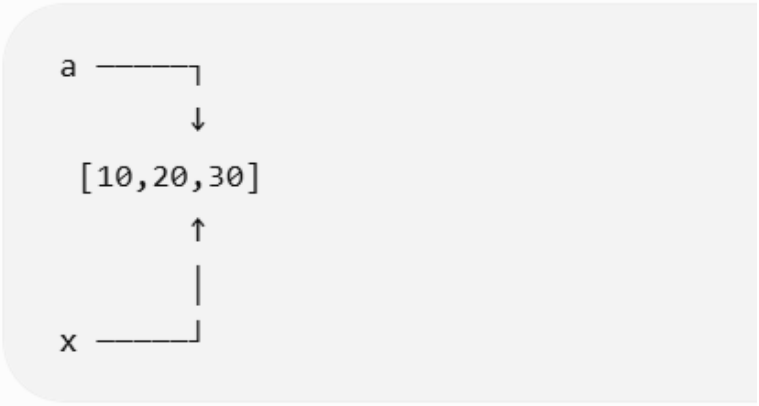
Python

Run

```python
def change(x):
    x = [100, 200]

a = [10, 20, 30]

change(a)

print(a)                    # Output: [10, 20, 30]
```

Kyun?

Initially:



Function mein:

```python
x = [100, 200]
```

ab `x` ko **new list** se bind kar diya:

```
a → [10,20,30]

x → [100,200]
```

Original `a` unchanged.

python

Concept	Python behavior
Pass by value	❌ Pure traditional sense mein nahi
Pass by reference	❌ C++ reference jaisa exactly nahi
Pass by object reference	✅ Python ka actual behavior
Mutable object modify	✅ Original object change ho sakta hai
Immutable object modify	❌ Original change nahi hota
Parameter ko reassign	❌ Original variable change nahi hota

-e7334f298d6c

List

Python

```python
def show(data):
    print(data)

my_list = [10, 20, 30]

show(my_list)

# Output: [10, 20, 30]
```

Tuple

Python

```python
def show(data):
    print(data)

my_tuple = (10, 20, 30)

show(my_tuple)
```

Set

Python

```python
def show(data):
    print(data)

my_set = {10, 20, 30}

show(my_set)

# Output: {10, 20, 30}
```

Dictionary

Python

```python
def show(data):
    print(data)

my_dict = {"name": "Tarun", "age": 22}

show(my_dict)
```

2. Ek hi function mein List + Tuple + Set + Dict

Bilkul pass kar sakte ho:

Python

```python
def show(list_data, tuple_data, set_data, dict_data):

    print(list_data)
    print(tuple_data)
    print(set_data)
    print(dict_data)


my_list = [10, 20, 30]

my_tuple = (40, 50, 60)

my_set = {70, 80, 90}

my_dict = {
    "name": "Tarun",
    "age": 22
}
```

```
show(my_list, my_tuple, my_set, my_dict)

# Output:
# [10, 20, 30]
# (40, 50, 60)
# {70, 80, 90}
# {'name': 'Tarun', 'age': 22}
```

Mapping समझो:

```
my_list   → list_data
my_tuple  → tuple_data
my_set    → set_data
my_dict   → dict_data
```

3. Ek function mein mixed data भी pass कर सकते हो

Python में parameter का type पहले से declare करना जरूरी नहीं है:

Python

```python
def show(a, b, c, d):

    print(type(a))
    print(type(b))
    print(type(c))
    print(type(d))


show(
    [1, 2, 3],
    (4, 5, 6),
    {7, 8, 9},
    {"name": "Tarun"}
)

# Output:
# <class 'list'>
# <class 'tuple'>
# <class 'set'>
```

4. Ek hi variable mein चारों को भी रख सकते हो

और ये भी possible है:

Python

```python
data = [
    [1, 2, 3],              # list
    (4, 5, 6),              # tuple
    {7, 8, 9},              # set
    {"name": "Tarun"}       # dictionary
]

def show(data):
    print(data)

show(data)

# Output:
# [[1, 2, 3], (4, 5, 6), {7, 8, 9}, {'name': 'Tarun'}]
```

यहाँ outer `data` एक **list** है, जिसके अंदर अलग-अलग types के objects हैं।

🔥 Function में कितने भी types pass कर सकते हो

Python dynamically typed language है:

Python

```python
def show(a, b, c, d, e):

    print(a)
    print(b)
    print(c)
    print(d)
    print(e)


show(
    10,                 # int
    "Hello",            # str
    [1, 2],             # list
    (3, 4),             # tuple
    {"x": 5}            # dict
)
```

python

Bilkul. Python **list ke important methods** ko table mein dekho:

Method	Kaam	Example	Result
`append(x)`	End mein 1 element add	`a.append(5)`	`[1,2,3,5]`
`extend(iterable)`	Multiple elements add	`a.extend([4,5])`	`[1,2,3,4,5]`
`insert(i,x)`	Given index par element add	`a.insert(1,10)`	`[1,10,2,3]`
`remove(x)`	First matching value delete	`a.remove(2)`	`[1,3]`
`pop()`	Last element delete + return	`a.pop()`	Last element
`pop(i)`	Given index delete + return	`a.pop(1)`	Index 1 element
`clear()`	Puri list empty	`a.clear()`	`[]`
`index(x)`	Value ka first index	`a.index(20)`	`1`
`count(x)`	Value kitni baar hai	`a.count(2)`	`2`
`sort()`	Ascending sort	`a.sort()`	`[1,2,3]`
`sort(reverse=True)`	Descending sort	`a.sort(reverse=True)`	`[3,2,1]`
`reverse()`	List ko reverse karta hai	`a.reverse()`	`[3,2,1]`
`copy()`	List ki shallow copy	`b=a.copy()`	New list

f

1. Built-in Functions vs List Methods

Function / Method	Type	Kisliye use hota hai	Example	Output
`dict()`	Built-in	Dictionary banane ke liye	`dict(name="Tarun")`	`{'name': 'Tarun'}`
`abs()`	Built-in	Absolute value	`abs(-5)`	`5`
`pow()`	Built-in	Power calculate	`pow(2, 3)`	`8`
`round()`	Built-in	Number round karna	`round(3.6)`	`4`
`sum()`	Built-in	Elements ka sum	`sum([1,2,3])`	`6`
`min()`	Built-in	Minimum value	`min([3,1,2])`	`1`
`max()`	Built-in	Maximum value	`max([3,1,2])`	`3`
`range()`	Built-in	Numbe uence	`list(range(5))`	`[0,1,2,3,4]`
`max()`	Built-in	Maximum value	`max([3,1,2])`	`3`
`range()`	Built-in	Number sequence	`list(range(5))`	`[0,1,2,3,4]`
`enumerate()`	Built-in	Index + value	`list(enumerate(['a','b']))`	`[(0,'a'),(1,'b')]`
`zip()`	Built-in	Multiple lists ko pair karna	`list(zip([1,2],['a','b']))`	`[(1,'a'),(2,'b')]`
`sorted()`	Built-in	Sorted **new list** banana	`sorted([3,1,2])`	`[1,2,3]`

Function	Kaam	Example	Output
`range()`	Number sequence	`list(range(5))`	`[0,1,2,3,4]`
`enumerate()`	Index + value	`list(enumerate(["a","b"]))`	`[(0,"a"),(1,"b")]`
`zip()`	Multiple iterables ko pair/combine	`list(zip([1,2],["a","b"]))`	`[(1,"a"),(2,"b")]`
`iter()`	Iterable se iterator banata hai	`it = iter([10,20])`	iterator
`next()`	Iterator ka next element	`next(it)`	`10`
`reversed()`	Reverse iterator	`list(reversed([1,2,3]))`	`[3,2,1]`
`sorted()`	Sorted **new list**	`sorted([3,1,2])`	`[1,2,3]`
`filter()`	Condition ke according elements	`list(filter(lambda x:x>2,[1,2,3,4]))`	`[3,4]`
`map()`	Har element par function apply	`list(map(lambda x:x*2,[1,2,3]))`	`[2,4,6]`
`all()`	Sab elements truthy hain?	`all([True,True,True])`	`True`
`any()`	Koi ek truthy hai?	`any([False,True,False])`	`True`

gg

Python में Important Functions/Loops — Complete Table

Function / Loop	Original list change?	Return क्या करता है?	Example
`for`	❌ No*	कोई value return नहीं	`for x in nums:`
`map()`	❌ No	`map object`	`map(lambda x:x*2, nums)`
`filter()`	❌ No	`filter object`	`filter(lambda x:x>2, nums)`
`reduce()`	❌ No	**Single final value**	`reduce(lambda a,b:a+b, nums)`
`all()`	❌ No	`True / False`	`all(x>0 for x in nums)`
`any()`	❌ No	`True / False`	`any(x>5 for x in nums)`
`sorted()`	❌ No	**New sorted list**	`sorted(nums)`
`reversed()`	❌ No	`erse iterator`	`reversed(nums)`

`sorted()`	❌ No	New sorted list	`sorted(nums)`
`reversed()`	❌ No	`reverse iterator`	`reversed(nums)`
`enumerate()`	❌ No	`enumerate object`	`enumerate(nums)`
`zip()`	❌ No	`zip object`	`zip(a,b)`
`range()`	❌ No	`range object`	`range(5)`
`list()`	Original object नहीं बदलता	**New list**	`list(range(5))`
`set()`	Original object नहीं बदलता	**New set**	`set(nums)`
`tuple()`	Original object नहीं बदलता	**New tuple**	`tuple(nums)`

python

```python
from functools import reduce

numbers = [1, 2, 3, 4, 5]

# map() → हर element पर operation
result = map(lambda x: x * 2, numbers)
print(list(result))                          # [2, 4, 6, 8, 10]

# filter() → condition के according elements
result = filter(lambda x: x > 2, numbers)
print(list(result))                          # [3, 4, 5]

# reduce() → सभी elements को combine करके ONE result
result = reduce(lambda a, b: a + b, numbers)
print(result)                                # 15

# all() → क्या सभी elements condition satisfy करते हैं?
result = all(x > 0 for x in numbers)
print(result)                                # True

# any() → क्या कम से कम एक element conditi  atisfy करता है?
result = any(x > 4 for x in numbers)
```

Python

```python
# any() → क्या कम से कम एक element condition satisfy करता है?
result = any(x > 4 for x in numbers)
print(result)                                   # True

# for → हर element पर operation (Python में forEach नहीं होता)
for x in numbers:
    print(x * 2, end=" ")                       # 2 4 6 8 10
```

एक ही नजर में 🧠

```
map()     → [2, 4, 6, 8, 10]    → map object
filter()  → [3, 4, 5]            → filter object
reduce()  → 15                   → single value
all()     → True                 → boolean
any()     → True                 → boolean
for       → 2 4 6 8 10           → कोई return नहीं
```

e7334f298d6c

1. Normal function पहले समझो

हम normally function ऐसे बनाते हैं:

Python

```python
def square(x):
    return x * x

print(square(5))       # 25
```

यहाँ:

```
square
  ↓
function का नाम
  ↓
function object
  ↓
जब square(5) करते हैं
  ↓
function execute होता है
  ↓
25
```

अब यही काम lambda से:

Python

```python
square = lambda x: x * x

print(square(5))       # 25
```

दोनों का basic काम same है।

---

2. Lambda function क्या है?

Lambda एक **small anonymous function** बनाने का तरीका है।

Syntax:

Python

```python
lambda arguments: expression
```

3. इसे "anonymous function" क्यों कहते हैं?

क्योंकि lambda बनाते समय normally इसका कोई नाम नहीं होता।

देखो:

Python

```
lambda x: x * 2
```

इसमें कोई नाम नहीं है जैसे:

Python

```
def double(x):
```

लेकिन अगर हम इसे variable में store कर दें:

Python

```
double = lambda x: x * 2
```

तो अब `double` variable उस lambda function object को refer कर रहा है।

तो अब `double` variable उस lambda function object को refer कर रहा है।

```
lambda x: x * 2
      ↓
  function object
      ↑
      |
   double
```

इसलिए technically **lambda function anonymous होता है**, लेकिन हम उसके object को variable में store करके नाम से use कर सकते हैं।

4. Lambda object क्या होता है?

यह सबसे important concept है।

जब Python यह code देखता है:

```python
lambda x: x * 2
```

Python internally **एक function object create करता है।**

Example:

```python
f = lambda x: x * 2
```

यहाँ:

b-e7334f298d6c

यहाँ:

```
lambda x: x * 2
        ↓
   Function Object
        ↑
        |
        f
```

`f` खुद function नहीं है।

Technically:

> `f` एक **reference/variable** है जो function object को point कर रहा है।

---

7. Normal `def` और lambda की class same होती है

देखो:

Python

```python
def add(a, b):
    return a + b

multiply = lambda a, b: a * b

print(type(add))
print(type(multiply))
```

Output:

```
<class 'function'>
<class 'function'>
```

Point	`def`	`lambda`
Function बनाता है?	✅	✅
Function object बनता है?	✅	✅
Type	`function`	`function`
Anonymous?	❌	✅
Name	Usually होता है	Initially नहीं
Multiple statements	✅	❌
`return` keyword	Usually use करते हैं	❌ नहीं
एक expression	जरूरी नहीं	✅
बड़े function के लिए	✅	❌
छोटे function के लिए	✅	✅

9. Lambda में `return` क्यों नहीं लिखते?

Normal function:

Python

```python
def square(x):
    return x * x
```

Lambda:

Python

```python
square = lambda x: x * x
```

9. Lambda में `return` क्यों नहीं लिखते?

Normal function:

```python
def square(x):
    return x * x
```

Lambda:

```python
square = lambda x: x * x
```

यहाँ:

```
x * x
```

का result automatically return होता है।

11. लेकिन expression complex हो सकता है

Lambda में conditional expression use कर सकते हैं:

```python
check = lambda x: "Even" if x % 2 == 0 else "Odd"

print(check(10))       # Even
print(check(7))        # Odd
```

यह एक expression है:

```
"Even" if condition else "Odd"
```

12. Multiple parameters भी हो सकते हैं

Python

```python
add = lambda a, b: a + b

print(add(10, 20))     # 30
```

तीन parameters:

Python

```python
calculate = lambda a, b, c: a + b + c

print(calculate(10, 20, 30))     # 60
```

13. Default parameter भी हो सकता है

Python

```python
greet = lambda name="Tarun": "Hello " + name

print(greet())              # Hello Tarun
print(greet("Rahul"))       # Hello Rahul
```

---

14. Lambda को variable में store कर सकते हैं

Python

```python
square = lambda x: x * x

print(square(4))            # 16
```

यहाँ:

:b-e7334f298d6c

15. Lambda को सीधे call भी कर सकते हैं

आपको variable बनाना जरूरी नहीं है।

Python

```
print((lambda x: x * 2)(5))
```

Output:

```
10
```

इसको break करके देखो:

Python

```
(lambda x: x * 2)(5)
```

16. Lambda का `__name__` क्या होता है?

Python

```python
f = lambda x: x * 2

print(f.__name__)
```

Output:

```
<lambda>
```

Normal function:

Python

```python
def square(x):
    return x * x

print(square.__name__)
```

17. Lambda object क अदर क्या-क्या हाता ह?

Lambda एक normal Python function object है, इसलिए इसमें function की बहुत सारी properties होती हैं।

```python
f = lambda x: x * 2

print(type(f))          # <class 'function'>
print(f.__name__)       # <lambda>
print(f.__module__)     # __main__
print(callable(f))      # True
```

सबसे important:

```python
callable(f)
```

18. `lambda object` और `map object` में confusion मत करना

यह बहुत important है क्योंकि तुम अभी `map()` पढ़ रहे थे।

देखो:

```python
f = lambda x: x * 2

m = map(f, [1, 2, 3])

print(type(f))
print(type(m))
```

Output:

```
<class 'function'>
<class 'map'>
```

यानी:

यानी:

```
lambda
  ↓
function object


map()
  ↓
map object
```

पूरा flow:

Python

```python
f = lambda x: x * 2

m = map(f, [1, 2, 3])

print(list(m))
```

Output:

19. `filter()` में lambda

Python

```python
numbers = [1, 2, 3, 4, 5]

f = lambda x: x > 3

result = filter(f, numbers)

print(list(result))       # [4, 5]
```

यहाँ 3 अलग चीजें हैं:

```
f
↓
lambda का function object

numbers
↓
original list

result
↓
```

20. `reduce()` में lambda

Python

```python
from functools import reduce

numbers = [1, 2, 3, 4]

f = lambda a, b: a + b

result = reduce(f, numbers)

print(result)             # 10
```

Flow:

```
lambda
 ↓
function object
 ↓
reduce()
 ↓
एक-एक करके combine
 ↓
```

21. Lambda को argument के रूप में भी भेज सकते हैं

यह lambda का सबसे common use है।

Python

```python
def execute(function, value):
    return function(value)

result = execute(lambda x: x * 2, 10)

print(result)             # 20
```

यहाँ:

```
lambda x: x * 2
       ↓
function object
       ↓
execute() को argument
       ↓
function(value)
       ↓
20
```

22. Function को list में भी रख सकते हो

Python

```python
functions = [
    lambda x: x + 10,
    lambda x: x * 2,
    lambda x: x ** 2
]

print(functions[0](5))       # 15
print(functions[1](5))       # 10
print(functions[2](5))       # 25
```

यह दिखाता है कि functions भी objects हैं।

```
list
 ├── function object
 ├── function object
 └── function object
```

23. Lambda को "class" को technically कैसे check करें?

Python

```python
f = lambda x: x * 2

print(type(f))
print(f.__class__)
```

Output:

```
<class 'function'>
<class 'function'>
```

दोनों same हैं।

```
f
↓
lambda function object
↓
f.__class__
↓
function
```

24. `types.FunctionType` से भी check कर सकते हो

Python

```python
import types

f = lambda x: x * 2

print(isinstance(f, types.FunctionType))
```

Output:

```
True
```

इससे confirm होता है कि lambda object वास्तव में Python का **function object** है।

python

```python
from functools import reduce
import types

# Lambda function/object
square = lambda x: x * x

print(square(5))                    # 25
print(type(square))                 # <class 'function'>
print(square.__class__)             # <class 'function'>
print(square.__name__)              # <lambda>
print(callable(square))             # True
print(isinstance(square, types.FunctionType))  # True


# map() + lambda
numbers = [1, 2, 3, 4, 5]

result = map(lambda x: x * 2, numbers)

print(list(result))                 # [2, 4, 6, 8, 10]
```

python

```python
# filter() + lambda
result = filter(lambda x: x > 3, numbers)

print(list(result))                 # [4, 5]


# reduce() + lambda
result = reduce(lambda a, b: a + b, numbers)

print(result)                       # 15


# all() + lambda-like condition
result = all(x > 0 for x in numbers)

print(result)                       # True


# any() + lambda-like condition
result = any(x > 4 for x in numbers)

print(result)                       # True
```

Python List vs Tuple — 8 Important Interview Points

Upgrade Share ...

#	Point	List	Tuple	Kyu / Interview Reason
1	**Mutability**	Mutable — elements change kar sakte hain	Immutable — elements change nahi kar sakte	**Data change hona hai → List**, fixed rakhna hai → Tuple
2	**Syntax**	`[10, 20, 30]`	`(10, 20, 30)`	List `[]` use karti hai, Tuple `()` use karta hai
3	**Performance**	Generally tuple se thodi slower	Generally list se faster for iteration/access	**Fixed data → Tuple** performance/memory efficiency ke liye useful
4	**Memory**	Generally zyada memory leti hai	Generally kam memory leti hai	Large fixed collections ke liye Tuple useful ho sakta hai
5	**Methods**	Bahut saare methods: `append()`, `remove()`, `sort()`, etc	Mainly `count()` aur `index()`	Data ko frequently modify karna hai → List
6	**Data Nature**	Dynamic/changing data ke liye	Fixed/constant data ke liye	**Shopping cart → List**, **coordinates → Tuple**
7	**Dictionary Key**	List dictionary key nahi ban sakti	Tuple (agar hashable elements hon) key ban sakta hai	Tuple immutable hone ki wajah se hashable ho sakta hai
8	**Safety / Intent**	Data modify ho sakta hai	Data modification prevent karta hai	Programmer ko batana ho ki **"ye data fixed hai" → Tuple**

3. Real-life examples — Interview mein bol sakte ho

Upgrade

Situation	Best Choice	Reason
Student list	**List**	Students add/remove ho sakte hain
Shopping cart items	**List**	Items add/remove hote hain
Employee records being edited	**List**	Data modify ho sakta hai
Coordinates `(x, y)`	**Tuple**	Fixed pair
RGB values `(255, 0, 0)`	**Tuple**	Usually fixed representation
Days of week	**Tuple**	Fixed collection
Database record	**Tuple**	Record ko fixed represent karna useful
Configuration constants	**Tuple**	Accidentally modification prevent kar sakte

Haan. Python **set ke methods** list/tuple se zyada hain. Set **unordered, mutable aur unique elements** ka collection hai.

🔹 Set ke all important methods

Method	Kaam	Example	Result
`add(x)`	1 element add	`s.add(5)`	`{1,2,3,5}`
`update(iterable)`	Multiple elements add	`s.update([4,5])`	`{1,2,3,4,5}`
`remove(x)`	Element remove; nahi mila to error	`s.remove(2)`	`{1,3}`
`discard(x)`	Element remove; nahi mila to error nahi	`s.discard(2)`	`{1,3}`
`pop()`	Random/arbitrary element remove + return	`s.pop()`	Element

	mila to error nahi		
`pop()`	Random/arbitrary element remove + return	`s.pop()`	Element
`clear()`	Pura set empty	`s.clear()`	`set()`
`copy()`	Shallow copy	`b=s.copy()`	New set

Example

Python

Run

```
s = {1, 2, 3}

s.add(4)
print(s)
```

Python

```python
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a.union(b))
print(a.intersection(b))
print(a.difference(b))
print(a.symmetric_difference(b))
```

Output:

```
{1, 2, 3, 4, 5, 6}
{3, 4}
{1, 2}
{1, 2, 5, 6}
```

f

python

Method	Operator	Kaam	Result
`union()`	`|`	Dono sets ke saare unique elements	`{1,2,3,4,5,6}`
`intersection()`	`&`	Common elements	`{3,4}`
`difference()`	`-`	`a` mein hain, `b` mein nahi	`{1,2}`
`symmetric_difference()`	`^`	Common ko chhodkar baaki	`{1,2,5,6}`

```python
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}


# 1. update()
x = s1.copy()
x.update(s2)
# s1 और s2 के सभी elements रखे जाएंगे (Union)
print(x)                                    # Output: {1, 2, 3, 4, 5, 6}


# 2. intersection_update()
x = s1.copy()
x.intersection_update(s2)
# सिर्फ common elements रखे जाएंगे
print(x)                                    # Output: {3, 4}


# 3. difference_update()
x = s1.copy()
x.difference_update(s2)
# s1 में जो elements s2 में नहीं हैं, सिर्फ वही रहेंगे
print(x)                                    # Output: {1, 2}
```

Python

```python
# 4. symmetric_difference_update()
x = s1.copy()
x.symmetric_difference_update(s2)
# Common elements हटेंगे, बाकी elements रहेंगे
print(x)                                    # Output: {1, 2, 5, 6}


# Original sets अभी भी unchanged हैं
print(s1)                                   # Output: {1, 2, 3, 4}
print(s2)                                   # Output: {3, 4, 5, 6}
```

🧠 अब flow बहुत simple है

```
s1 = {1,2,3,4}        ← Original
s2 = {3,4,5,6}        ← Original

       ↓ copy()

x = {1,2,3,4}         ← Temporary copy

       ↓ operation
```

python

Method	Operator	क्या करता है?	`A` का result
`A.update(B)`	`|=`	दोनों sets के सभी elements रखता है	`{1,2,3,4,5,6}`
`A.intersection_update(B)`	`&=`	सिर्फ common elements रखता है	`{3,4}`
`A.difference_update(B)`	`-=`	`A` में जो `B` में नहीं हैं, वही रखता है	`{1,2}`
`A.symmetric_difference_update(B)`	`^=`	Common हटाता है, बाकी रखता है	`{1,2,5,6}`

⭐ Set ke methods ek jagah

```
add()
update()

remove()
discard()
pop()
clear()
copy()

union()
intersection()
difference()
symmetric_difference()

issubset()
issuperset()
isdisjoint()

intersection_update()
difference_update()
symmetric_difference_update()
```

🔵 Python में Set क्या है?

Definition

> **Set Python ka ek built-in collection data type hai jo multiple elements ko store karta hai, jisme duplicate elements allowed nahi hote aur set unordered hota hai.**

Simple language mein:

> **Set = unique values ka collection.**

Example:

Python

Run

```
s = {10, 20, 30, 40}

print(s)
```

Output:

Property	Set में
Ordered	❌ No
Duplicate allowed	❌ No
Mutable	✅ Yes
Indexing	❌ No
Slicing	❌ No
Multiple data types	✅ Yes
Hashable elements	✅ Required
`{}`	❌ Empty set नहीं
`set()`	✅ Empty set
`in` operator	✅
`not in`	✅

🔵 Frozenset क्या है?

Definition

> `frozenset` Python का एक immutable version of `set` है।

Simple Hinglish:

> Set में elements add/remove/update कर सकते हैं, लेकिन frozenset बनने के बाद उसके elements को change नहीं कर सकते।

Example:

Python

Run

```python
s = {1, 2, 3}

fs = frozenset([1, 2, 3])

print(s)       # {1, 2, 3}
print(fs)      # frozenset({1, 2, 3})
```

2. Frozenset कैसे बनाते हैं?

`frozenset()` constructor से:

Python

```
fs = frozenset([1, 2, 3, 4])

print(fs)
```

Output:

```
frozenset({1, 2, 3, 4})
```

Tuple से:

Python

```
fs = frozenset((10, 20, 30))

print(fs)
```

String से:

```python
fs = frozenset("hello")

print(fs)
```

Possible output:

```
frozenset({'h', 'e', 'l', 'o'})
```

Duplicate automatically remove होंगे।

5. Frozenset immutable है — इसका actual मतलब

Immutable का मतलब सिर्फ यह नहीं कि `add()` नहीं कर सकते।

इसका मतलब:

> **Frozenset object बनने के बाद उसके अंदर के elements को modify, add या delete नहीं किया जा सकता।**

ये सारे operations नहीं कर सकते:

Python

Run

```python
fs = frozenset([1, 2, 3])

fs.add(4)          # ❌
fs.remove(2)       # ❌
fs.discard(2)      # ❌
fs.pop()           # ❌
fs.clear()         # ❌
fs.update([4, 5])  # ❌
```

python

```python
s1 = frozenset([1, 2, 3, 4])
s2 = frozenset([3, 4, 5, 6])


# 1. add()
# Frozenset immutable hai, isliye add() method nahi hota
# s1.add(7)                          # ❌ AttributeError


# 2. union()
result = s1.union(s2)
print(result)                        # Output: frozenset({1, 2, 3, 4, 5, 6})
# s1 aur s2 ke saare unique elements


# 3. intersection()
result = s1.intersection(s2)
print(result)                        # Output: frozenset({3, 4})
# s1 aur s2 ke common elements
```

python

```python
# 4. difference()
result = s1.difference(s2)
print(result)                          # Output: frozenset({1, 2})
# s1 mein hain, lekin s2 mein nahi hain


# 5. symmetric_difference()
result = s1.symmetric_difference(s2)
print(result)                          # Output: frozenset({1, 2, 5, 6})
# Common elements hatao, baaki dono sets ke elements rakho


# Original frozensets unchanged
print(s1)                              # Output: frozenset({1, 2, 3, 4})
print(s2)                              # Output: frozenset({3, 4, 5, 6})
```

SET

→ Unique

→ Unordered

→ Mutable

→ Not Hashable

FROZENSET

→ Unique

→ Unordered

→ Immutable

→ Hashable

> **Frozenset ka ek fixed hash value ban sakta hai, isliye usse dictionary ki key ya kisi doosre set ke element ke रूप में use kar sakte hain.**

Pehle `hash` kya hota hai?

Python kisi object se ek **integer value** generate kar sakta hai:

Python

Run

```python
x = frozenset([1, 2, 3])

print(hash(x))       # Example: 529344067295497451
```

Ye number Python ko object ko efficiently identify/store karne mein help karta hai.

🔥 Frozenset hashable kyu hai?

Frozenset **immutable** hai.

```
frozenset
    ↓
change nahi kar sakte
    ↓
uski value stable rahti hai
    ↓
hash stable reh sakta hai
    ↓
HASHABLE ✅
```

Lekin normal `set`:

```
set
 ↓
mutable
 ↓
elements change ho sakte hain
 ↓
hash stable nahi rahega
 ↓
```

1. Frozenset ko dictionary key bana sakte hain

Python

```python
permissions = frozenset(["read", "write"])

data = {
    permissions: "Editor"
}

print(data)
```

Output:

```
{frozenset({'read', 'write'}): 'Editor'}
```

Ye possible hai kyunki `frozenset` hashable hai.

3. Frozenset ko set ke andar bhi rakh sakte hain

Python

```python
a = frozenset([1, 2])
b = frozenset([3, 4])

s = {a, b}

print(s)
```

Possible output:

```
{frozenset({1, 2}), frozenset({3, 4})}
```

Dictionary Methods

Method	Kaam	Example	Result
`clear()`	Puri dictionary empty	`d.clear()`	`{}`
`copy()`	Shallow copy	`d2 = d.copy()`	New dict
`fromkeys()`	Given keys se new dict	`dict.fromkeys(["a","b"], 0)`	`{'a':0,'b':0}`
`get()`	Key ki value safely get	`d.get("name")`	Value / `None`
`items()`	Key-value pairs deta hai	`d.items()`	`dict_items(...)`
`keys()`	Saari keys	`d.keys()`	`dict_keys(...)`
`pop()`	Given key delete + value return	`d.pop("age")`	Value
`popitem()`	Last inserted key-value delete + return	`d.popitem()`	`(key,value)`
`popitem()`	Last inserted key-value delete + return	`d.popitem()`	`(key,value)`
`setdefault()`	Key exist na ho to add karta hai	`d.setdefault("city","Indore")`	Value
`update()`	Dictionary update/add	`d.update({"age":22})`	Updated dict
`values()`	Saari values	`d.values()`	`dict_values(...)`

python

```python
# Original Dictionary
d = {
    "name": "Tarun",
    "age": 22,
    "city": "Indore"
}


# 1. clear()
x = d.copy()
result = x.clear()
print(x)                                # Output: {}
print(result)                           # Output: None
# clear() पूरी dictionary को empty कर देता है


# 2. copy()
d2 = d.copy()
print(d2)                               # Output: {'name': 'Tarun', 'age': 22, 'city': 'Indore
# copy() dictionary की नई/shallow copy बनाता है


# 3. fromkeys()
```

python

```python
# 3. fromkeys()
keys = ["a", "b", "c"]
result = dict.fromkeys(keys, 0)
print(result)                          # Output: {'a': 0, 'b': 0, 'c': 0}
# दिए गए सभी keys की नई dictionary बनती है और सभी की value 0 है


# 4. get()
result = d.get("name")
print(result)                          # Output: Tarun
# मौजूद key की value देता है

result = d.get("salary")
print(result)                          # Output: None
# key नहीं मिली तो get() error नहीं देता, None देता है


# 5. items()
result = d.items()
print(result)                          # Output: dict_items([('name', 'Tarun'), ('age', 22),
# सभी key-value pairs देता है
```

python

```python
# 6. keys()
result = d.keys()
print(result)                          # Output: dict_keys(['name', 'age', 'city'])
# सभी keys देता है


# 7. pop()
x = d.copy()
result = x.pop("age")
print(result)                          # Output: 22
print(x)                               # Output: {'name': 'Tarun', 'city': 'Indore'}
# 'age' key delete हुई और उसकी value 22 return हुई


# 8. popitem()
x = d.copy()
result = x.popitem()
print(result)                          # Output: ('city', 'Indore')
print(x)                               # Output: {'name': 'Tarun', 'age': 22}
# Last inserted key-value pair delete करके tuple के रूप में return करता है
```

python

```python
# 1. popitem()
x = d.copy()

result = x.popitem()

print(result)                          # Output: ('city', 'Indore')
print(x)                               # Output: {'name': 'Tarun', 'age': 22}
# Last inserted key-value pair delete करता है
# और deleted pair को (key, value) tuple के रूप में return करता है



# 2. setdefault()
x = d.copy()

result = x.setdefault("city", "Bhopal")

print(result)                          # Output: Indore
print(x)                               # Output: {'name': 'Tarun', 'age': 22, 'city': 'Indore
# 'city' पहले से मौजूद है → उसकी existing value return होगी
# Dictionary में कोई change नहीं होगा
```

python

```python
# setdefault() जब key मौजूद नहीं हो
x = d.copy()

result = x.setdefault("salary", 50000)

print(result)                          # Output: 50000
print(x)                               # Output: {'name': 'Tarun', 'age': 22, 'city': 'Indore
# 'salary' मौजूद नहीं थी → नई key add हुई और value 50000 मिली


# 3. update()
x = d.copy()

result = x.update({"age": 25, "salary": 50000})

print(result)                          # Output: None
print(x)                               # Output: {'name': 'Tarun', 'age': 25, 'city': 'Indore
# Existing key 'age' की value update हुई
# New key 'salary' add हुई
# update() original dictionary को modify करता है और None return करता है
```

Python Run

```python
# 4. values()
result = d.values()

print(result)                          # Output: dict_values(['Tarun', 22, 'Indore'])
# Dictionary की सभी values देता है
```

🧠 चारों को एकदम simple रखो

Method	काम	Dictionary change?	Return
`popitem()`	Last key-value delete	✅	`(key, value)`
`setdefault()`	Key नहीं है तो add करता है	कभी-कभी ✅	Value
`update()`	Existing key update + नई key add	✅	`None`
`values()`	सभी values देता है	❌	`dict_values`

Method	क्या करता है?	Return
`clear()`	पूरी dictionary empty	`None`
`copy()`	नई shallow dictionary बनाता है	New `dict`
`fromkeys()`	Given keys से नई dictionary	New `dict`
`get()`	Key की value निकालता है	Value / `None`
`items()`	Key-value pairs देता है	`dict_items`
`keys()`	सभी keys देता है	`dict_keys`
`pop()`	Given key delete करता है	Deleted value
`popitem()`	Last inserted pair delete करता है	`(key, value)`

Upgrade Share ...

8 Important Differences

#	Point	Set	Dictionary	Kyu / Kab use karein?
1	**Definition**	Unique elements ka unordered collection	Key-value pairs ka collection	Sirf unique data chahiye → **Set**; data ke saath information associate karni ho → **Dictionary**
2	**Syntax**	`{10, 20, 30}`	`{"name": "Tarun", "age": 22}`	Set mein values; Dictionary mein `key: value`
3	**Duplicates**	❌ Duplicate values allowed nahi	❌ Duplicate **keys** allowed nahi, values duplicate ho sakti hain	Unique items → Set; unique identifier → Dictionary key
4	**Data Structure**	`value`	`key → value`	Set mein direct value; Dictionary mein key ke through value access
5	**Access**	Indexing na[illegible]oti	Key ke through value access	`set[0]` ❌, `dict["name"]` ✅
6	**Main Purpose**	Uniqueness aur set operations	Data ko key ke saath map/store karna	Duplicate remove → Set; record/data mapping → Dictionary
7	**Search**	`x in set` generally very fast	`key in dict` generally very fast	Membership/lookup ke liye dono useful
8	**Operations**	Union, intersection, difference	Add/update/delete key-value pairs	Mathematical set operations → Set; structured records → Dictionary

Mutable: `List + Dict + Set + Bytearray`

Immutable: `int + float + complex + bool + str + tuple + frozenset + bytes + None`

8. Important format specifiers

Specifier	Meaning	Example	Output
`.2f`	2 decimal places	`f"{5.678:.2f}"`	`5.68`
`.3f`	3 decimal places	`f"{5.6789:.3f}"`	`5.679`
`d`	Integer	`f"{10:d}"`	`10`
`,`	Thousands separator	`f"{1000000:,}"`	`1,000,000`
`%`	Percentage	`f"{0.75:.2%}"`	`75.00%`
`>10`	Right align	`f"{'Hi':>10}"`	right aligned
`<10`	Left align	`f"{'Hi':<10}"`	left aligned
`^10`	Center align	`f"{'Hi':^10}"`	centered

python

```python
# 1. .2f → 2 decimal places
num = 5.678
print(f"{num:.2f}")                 # Output: 5.68


# 2. .3f → 3 decimal places
num = 5.6789
print(f"{num:.3f}")                 # Output: 5.679


# 3. d → Integer
num = 10
print(f"{num:d}")                   # Output: 10


# 4. , → Thousands separator
num = 1000000
print(f"{num:,}")                   # Output: 1,000,000


# 5. % → Percentage
num = 0.75
```

python

```python
# 5. % → Percentage
num = 0.75
print(f"{num:.2%}")                 # Output: 75.00%


# 6. >10 → Right alignment (total width = 10)
text = "Hi"
print(f"{text:>10}")                # Output:         Hi


# 7. <10 → Left alignment (total width = 10)
text = "Hi"
print(f"{text:<10}")                # Output: Hi


# 8. ^10 → Center alignment (total width = 10)
text = "Hi"
print(f"{text:^10}")                # Output:    Hi
```

Python

```python
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print(A ^ B)
```

Result:

```
{1, 2, 5, 6}
```

Difference vs Symmetric Difference

Operation	Meaning	Result
`A - B`	A mein, B mein nahi	`{1,2}`
`B - A`	B mein, A mein nahi	`{5,6}`
`A ^ B`	Dono ke unique, common nahi	`{1,2,5,6}`

Upgrade

Python String Methods

Method	Kaam	Example
`capitalize()`	First character uppercase	`"hello".capitalize()` → `"Hello"`
`casefold()`	Strong lowercase conversion	`"HELLO".casefold()` → `"hello"`
`lower()`	Lowercase	`"HELLO".lower()` → `"hello"`
`upper()`	Uppercase	`"hello".upper()` → `"HELLO"`
`title()`	Har word ka first letter uppercase	`"hello world".title()` → `"Hello World"`
`swapcase()`	Upper ↔ lower	`"HeLLo".swapcase()` → `"hEllO"`

Search / Find

Upgra

Method	Kaam	Example
`find()`	First occurrence ka index; nahi mila → `-1`	`"hello".find("l")` → `2`
`rfind()`	Right se search	`"hello".rfind("l")` → `3`
`index()`	First occurrence; nahi mila → error	`"hello".index("l")` → `2`
`rindex()`	Right se index	`"hello".rindex("l")` → `3`
`count()`	Occurrences count	`"hello".count("l")` → `2`
`startswith()`	Given prefix se start?	`"hello".startswith("he")` → `True`
`endswith()`	Given suffix se end?	`"hello".endswith("lo")` → `True`

python

Method	काम	Example	Output	Retu
`replace()`	String के text को replace करता है	`"hello".replace("l", "x")`	`"hexxo"`	`str`
`split()`	String को separator के आधार पर अलग करके list बनाता है	`"a,b,c".split(",")`	`['a', 'b', 'c']`	`list`
`rsplit()`	Right side से string को split करता है	`"a-b-c".rsplit("-", 1)`	`['a-b', 'c']`	`list`
`splitlines()`	String को lines के आधार पर split करता है	`"a\nb".splitlines()`	`['a', 'b']`	`list`
`partition()`	Separator के around **3-part tuple** बनाता है	`"a-b".partition("-")`	`('a', '-', 'b')`	`tuple`
`rpartition()`	Right side से separator के around **3-part tuple** बनाता है	`"a-b-c".rpartition("-")`	`('a-b', '-', 'c')`	`tuple`
`join()`	Iterable के elements को separator से जोड़कर string	`"-".join(["a", "b"])`	`"a-b"`	`str`

python

```python
# 1. replace() → string के text को replace करता है
result = "hello".replace("l", "x")
print(result)                          # Output: hexxo


# 2. split() → string को list में convert करता है
result = "a,b,c".split(",")
print(result)                          # Output: ['a', 'b', 'c']


# 3. rsplit() → right side से split करता है
result = "a-b-c".rsplit("-", 1)
print(result)                          # Output: ['a-b', 'c']


# 4. splitlines() → अलग-अलग lines को list में रखता है
result = "a\nb".splitlines()
print(result)                          # Output: ['a', 'b']
```

```python
# 5. partition() → string को 3 parts के tuple में divide करता है
result = "a-b".partition("-")
print(result)                          # Output: ('a', '-', 'b')


# 6. rpartition() → right side से partition करता है
result = "a-b-c".rpartition("-")
print(result)                          # Output: ('a-b', '-', 'c')


# 7. join() → iterable के elements को जोड़कर एक string बनाता है
result = "-".join(["a", "b"])
print(result)                          # Output: a-b
```

🔹 Remove spaces / characters

Method	Kaam	Example
`strip()`	Left + right whitespace remove	`" hi ".strip()` → `"hi"`
`lstrip()`	Left whitespace	`" hi".lstrip()`
`rstrip()`	Right whitespace	`"hi ".rstrip()`
`removeprefix()`	Starting prefix remove	`"Mr.Tarun".removeprefix("Mr.")`
`removesuffix()`	Ending suffix remove	`"file.txt".removesuffix(".txt")`

🔹 Checking methods ⭐

Ye methods **True/False** return karte hain.

Method	Check
`isalpha()`	Sirf alphabets?
`isdigit()`	Sirf digits?
`isdecimal()`	Decimal characters?
`isnumeric()`	Numeric characters?
`isalnum()`	Alphabet + number?
`isspace()`	Sirf whitespace?
`islower()`	Lowercase?
`isupper()`	Uppercase?

`isupper()`	Uppercase?
`istitle()`	Title case?
`isascii()`	ASCII characters?
`isidentifier()`	Valid Python identifier?
`isprintable()`	Printable characters?
`isdecimal()`	Decimal digits?
`isnumeric()`	Numeric characters?
`isidentifier()`	Valid identifier?

d01-8ee16ed71cff

⭐ String methods ek saath

```
capitalize()
casefold()
lower()
upper()
title()
swapcase()

find()
rfind()
index()
rindex()
count()
startswith()
endswith()

replace()
split()
rsplit()
splitlines()
partition()
rpartition()
join()
```

-8ee16ed71cff

```
join()

strip()
lstrip()
rstrip()
removeprefix()
removesuffix()

isalpha()
isdigit()
isdecimal()
isnumeric()
isalnum()
isspace()
islower()
isupper()
istitle()
isascii()
isidentifier()
isprintable()

center()
ljust()
rjust()
zfill()
```

Upgrade Share

Mode	Meaning / Purpose	File already exist/present hai	File exist/present nahi hai	Example
`r`	Existing file read	✅ Open	❌ `FileNotFoundError`	`open("a.txt","r")`
`w`	Write; purana data overwrite	⚠️ Overwrite	✅ Create	`open("a.txt","w")`
`x`	**Sirf new file create**	❌ `FileExistsError`	✅ Create	`open("a.txt","x")`
`a`	File ke end mein data add	✅ Open	✅ Create	`open("a.txt","a")`
`r+`	Read + write, existing file	✅ Open	❌ `FileNotFoundError`	`open("a.txt","r+")`
`w+`	Write + read; overwrite	⚠️ Purana data delet	✅ Create	`open("a.txt","w+")`
	overwrite	delete		
`a+`	Append + read	✅ Open	✅ Create	`open("a.txt","a+")`
`rb`	**Binary read**	✅ Open	❌ Error	`open("photo.jpg","rb")`
`wb`	**Binary write/overwrite**	⚠️ Overwrite	✅ Create	`open("photo.jpg","wb")`
`ab`	**Binary append**	✅ Open	✅ Create	`open("photo.jpg","ab")`
`t`	**Text mode modifier**	—	—	`open("a.txt","rt")`
`b`	**Binary mode modifier**	—	—	`open("a.jpg","rb")`
`+`	**Read + write modifier**	Mode par depend	Mode par depend	`r+`, `w+`, `a+`

python

```python
with open("demo.txt", "w") as f:
    f.write("Hello")

with open("demo.txt", "r") as f:
    print(f.read())                          # Output: Hello


# ------------------------------------------------------------
# 2. "a" → Append
# Existing data ko delete nahi karta
# New data END mein add karta hai
# ------------------------------------------------------------

with open("demo.txt", "a") as f:
    f.write(" Python")

with open("demo.txt", "r") as f:
    print(f.read())                          # Output: Hello Python
```

python

```python
# 3. "r" → Read
# Existing file ko read karta hai
# File nahi hai → FileNotFoundError
# ------------------------------------------------------------

with open("demo.txt", "r") as f:
    print(f.read())                          # Output: Hello Python


# ------------------------------------------------------------
# 4. "x" → Create ONLY
# File already exist karti hai → FileExistsError
# ------------------------------------------------------------

with open("newfile.txt", "x") as f:
    f.write("New File")

with open("newfile.txt", "r") as f:
    print(f.read())                          # Output: New File
```

python

```python
# ------------------------------------------------------------
# 5. "r+" → Read + Write
# Existing file required
# Purana data automatically delete nahi hota
# ------------------------------------------------------------

with open("demo.txt", "r+") as f:
    print(f.read())                          # Output: Hello Python
    f.write("!")                             # End mein ! add


# ------------------------------------------------------------
# 6. "w+" → Write + Read
# File create karta hai
# Existing file ka purana data DELETE/OVERWRITE karta hai
# ------------------------------------------------------------

with open("wplus.txt", "w+") as f:
    f.write("ABC")
    f.seek(0)
    print(f.read())                          # Output: ABC
```

```python
# 7. "a+" → Append + Read
# File create bhi karta hai
# New data END mein add karta hai
# ------------------------------------------------------------

with open("aplus.txt", "a+") as f:
    f.write("Hello")
    f.seek(0)
    print(f.read())                          # Output: Hello


# ------------------------------------------------------------
# 8. "rb" → Binary Read
# Image/PDF/audio etc. binary data read
# ------------------------------------------------------------

with open("demo.txt", "rb") as f:
    data = f.read()

print(type(data))                            # Output: <class 'bytes'>
```

python

```python
# 9. "wb" → Binary Write
# Binary data write karta hai
# Existing data overwrite karta hai
# ------------------------------------------------------------

with open("binary.dat", "wb") as f:
    f.write(b"Hello")

with open("binary.dat", "rb") as f:
    print(f.read())                          # Output: b'Hello'


# ------------------------------------------------------------
# 10. "ab" → Binary Append
# Binary data END mein add karta hai
# ------------------------------------------------------------

with open("binary.dat", "ab") as f:
    f.write(b" Python")

with open("binary.dat", "rb") as f:
    print(f.read())                          # Output: b'Hello Python'
```

```python
# 11. "t" → Text mode
# Usually default mode hota hai
# ------------------------------------------------------------

with open("demo.txt", "rt") as f:
    data = f.read()

print(type(data))                          # Output: <class 'str'>


# ------------------------------------------------------------
# 12. "b" → Binary mode
# Binary data bytes ke form mein
# ------------------------------------------------------------

with open("binary.dat", "rb") as f:
    data = f.read()

print(type(data))                          # Output: <class 'bytes'>
```

1. `tell()` — Current position batata hai

File ke andar **cursor/file pointer abhi kis position par hai** ye batata hai.

2. `seek()` — Cursor ko kisi position par le jaata hai

Python

```python
file.seek(0)
```

Matlab cursor ko **starting position (0)** par le jao.

Ek hi code mein:

```
with open("data.txt", "r") as file:
    print("Position:", file.tell())
    print(file.read(5))
    print("Position:", file.tell())
    file.seek(0)
    print("Position:", file.tell())
    print(file.read(5))
```

Agar `data.txt` mein:

```
Hello Tarun
```

hai, output roughly:

```
Position: 0
Hello
Position: 5
Position: 0
Hello
```

Samjho: `tell()` = "Main kahan hoon?"

`seek()` = "Mujhe yahan le jao."

```python
with open("data.txt", "r+") as file:

    print("1. tell() =", file.tell())

    print("2. read(5) =", file.read(5))

    print("3. tell() =", file.tell())

    file.seek(0)
    print("4. seek(0) ke baad =", file.tell())

    print("5. readline() =", file.readline().strip())

    file.seek(0)
    print("6. read() =")
    print(file.read())

    print("7. readable() =", file.readable())
    print("8. writable() =", file.writable())
    print("9. seekable() =", file.seekable())
```

Output

```
1. tell() = 0
2. read(5) = Hello
3. tell() = 5
4. seek(0) ke baad = 0
5. readline() = Hello
6. read() =
Hello
Python
Tarun
7. readable() = True
8. writable() = True
9. seekable() = True
```

Method	Kya karta hai	Code mein	Output/Result
`open()`	File open karta hai	`open("data.txt", "r+")`	File open
`read()`	Puri remaining file read	`file.read()`	`Hello Python Tarun`
`read(n)`	`n` characters read	`file.read(5)`	`Hello`
`readline()`	Ek line read	`file.readline()`	`Hello`
`readlines()`	Saari lines ko list mein deta hai	`file.readlines()`	`['Hello\n', 'Python\n', 'Tarun']`
`write()`	Data file mein likhta hai	`file.write("ABC")`	Data add
`writelines()`	Multiple strings likhta hai	`file.writelines(["A\n","B\n"])`	Multiple lines
`tell()`	Cursor ki current position	`file.tell()`	`0`, `5`
`tell()`	Cursor ki current position	`file.tell()`	`0`, `5`
`seek()`	Cursor ko move karta hai	`file.seek(0)`	Cursor → beginning
`flush()`	Buffered data ko file mein push karta hai	`file.flush()`	Data immediately push
`close()`	File close karta hai	`file.close()`	File closed
`readable()`	File read ho sakti hai?	`file.readable()`	`True`
`writable()`	File write ho sakti hai?	`file.writable()`	`True`
`seekable()`	Cursor move ho sakta hai?	`file.seekable()`	`True`

Bilkul. **Python me access ke basis par** table se samjho:

Access Specifier	Syntax	Same Class	Child Class	Outside Class	Main Point
Public	`name`	✅	✅	✅	Kahin se bhi access
Protected	`_name`	✅	✅	⚠️ Possible	Python me restriction nahi, convention hai
Private	`__name`	✅	❌ Directly	❌ Directly	Name mangling hoti hai

```
class A:

    public = 10
    _protected = 20
    __private = 30

    def show(self):
        print(self.public)       # ✅
        print(self._protected)   # ✅
        print(self.__private)    # ✅


class B(A):

    def show(self):
        print(self.public)       # ✅
        print(self._protected)   # ✅
        # print(self.__private)  # ❌
```

Python में **class का object बनाने के लिए** `new` **keyword नहीं होता।**

Python

Python

```python
class Student:
    def __init__(self, name):
        self.name = name

s1 = Student("Tarun")
print(s1.name)
```

यहाँ:

Language	Object creation
Python	`s1 = Student("Tarun")`
Java	`Student s1 = new Student("Tarun");`
JavaScript	`let s1 = new Student("Tarun");`
C++	`Student s1("Tarun");` या `new Student("Tarun")`

Python में `self` **बहुत important है**, खासकर class के methods में। आसान भाषा में समझो:

1. `self` क्या है?

`self` current object **का** reference है।

Python

```python
class Student:
    def show(self):
        print("Hello")

s1 = Student()
s1.show()
```

जब:

Python

```python
s1.show()
```

चलता है, Python internally लगभग इसे ऐसे समझता है:

Python

```python
Student.show(s1)
```

इसलिए `self` के अंदर `s1` object **का** reference आता है।

2. `self` क्या करता ह?

सबसे important काम है **object के variables और methods को access करना।**

Python

```python
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def show(self):
        print(self.name)
        print(self.marks)

s1 = Student("Tarun", 85)
s2 = Student("Rahul", 90)

s1.show()
s2.show()
```

4. `self` लिख दिया लेकिन use नहीं किया?

यह valid है:

```python
class Student:
    def show(self):
        print("Hello")

s1 = Student()
s1.show()
```

Output:

```
Hello
```

`self` को method के अंदर use करना जरूरी **नहीं** है।

लेकिन method को object के साथ काम करना है तो `self` चाहिए।

5. `self` क्यों जरूरी है?

मान लो दो objects हैं:

Python

```python
class Student:
    def __init__(self, name):
        self.name = name

s1 = Student("Tarun")
s2 = Student("Rahul")
```

Memory को conceptually ऐसे समझो:

```
s1
 └── name = "Tarun"

s2
 └── name = "Rahul"
```

तो:

```
self → s1
self.name → "Tarun"
```

जब:

```python
s2.show()
```

तो:

```
self → s2
self.name → "Rahul"
```

इसलिए `self` बताता है कि **किस object की property पर काम करना है।**

6. `self.name` और `name` में difference

```python
class Student:
    def __init__(self, name):
        self.name = name
```

यहाँ:

Code	Meaning
`name`	function का local parameter
`self.name`	object का instance variable

`this` kis language mein hota hai?

Language	Current object ko refer karne ke liye
Python	`self`
Java	`this`
JavaScript	`this`
C++	`this`
C#	`this`

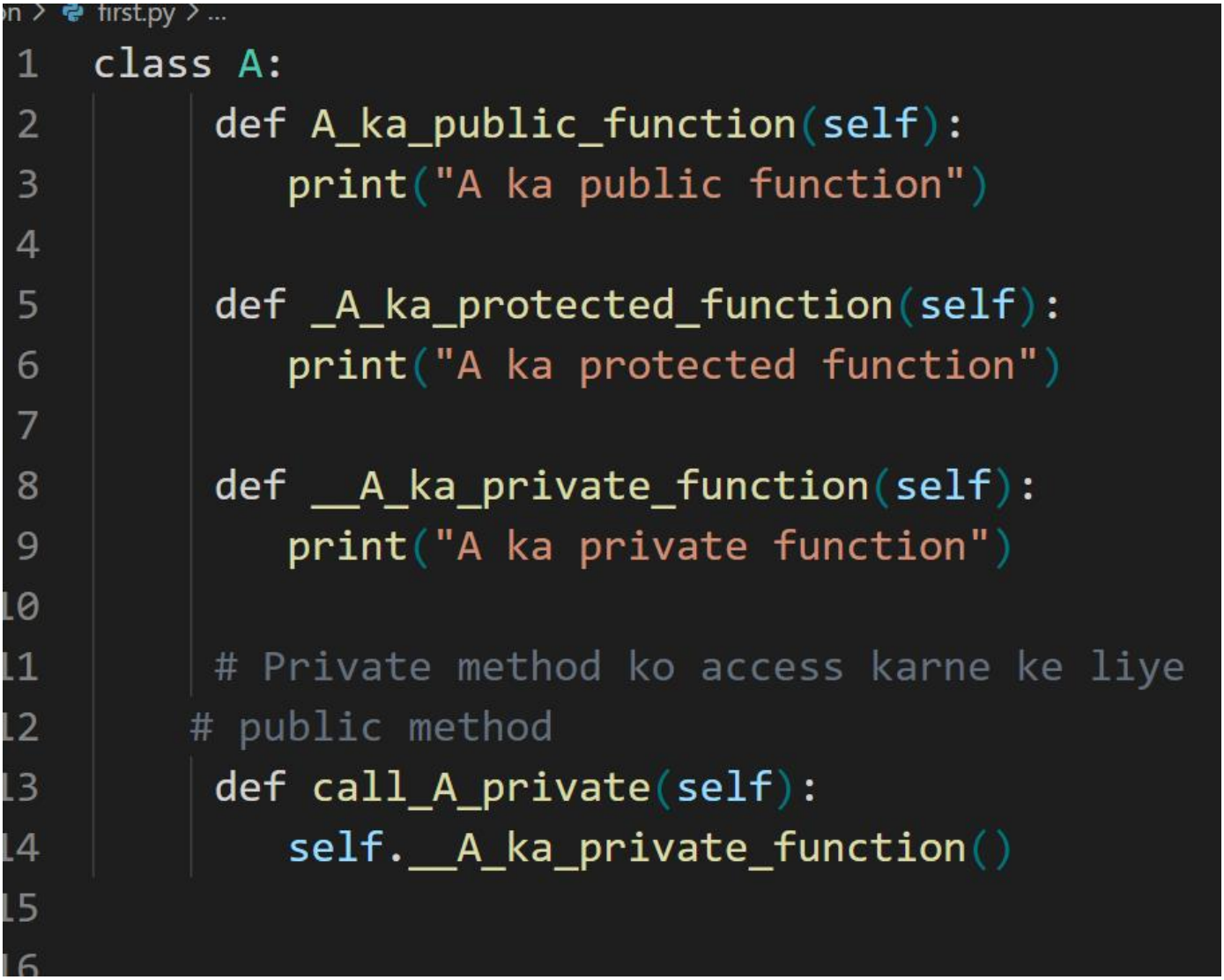
python

```python
class A:
    A_ka_public = "Tarun"
    _A_ka_protected = 22
    __A_ka_private = 50000
     # Private ko access karne ke liye method
    def get_A_private(self):
        return self.__A_ka_private


class B(A):
    B_ka_public = 44
    _B_ka_protected = "Tarun"
    __B_ka_private = 5.4
# B ke private ko access karne ke liye method
    def get_B_private(self):
        return self.__B_ka_private
```

python

```python
s1 = B()
print("A public:", s1.A_ka_public)
print("B public:", s1.B_ka_public)

print("A protected:", s1._A_ka_protected)
print("B protected:", s1._B_ka_protected)

# Direct access ❌
# print(s1.__A_ka_private)
# print(s1.__B_ka_private)

# Method ke through access ✅
print("A private:", s1.get_A_private())
print("B private:", s1.get_B_private())
```

f


first.py > ...
class A:
def A_ka_public_function(self):
print("A ka public function")
def _A_ka_protected_function(self):
print("A ka protected function")
def __A_ka_private_function(self):
print("A ka private function")
# Private method ko access karne ke liye
# public method
def call_A_private(self):
self.__A_ka_private_function()

```python
class B(A):
        def B_ka_public_function(self):
         print("B ka public function")

        def _B_ka_protected_function(self):
         print("B ka protected function")

        def __B_ka_private_function(self):
         print("B ka private function")

    # Private method ko access karne ke liye
    # public method
        def call_B_private(self):
         self.__B_ka_private_function()
```

python

```python
s1 = B()
s1.A_ka_public_function()        # A se inherited
s1.B_ka_public_function()        # B ka


s1._A_ka_protected_function()    # A se inherited
s1._B_ka_protected_function()    # B ka

# Direct access ❌

# s1.__A_ka_private_function()
# s1.__B_ka_private_function()

# Method ke through access ✅
s1.call_A_private()
s1.call_B_private()
```

python

```python
class A:

    # ========================
    # CONSTRUCTOR
    # ========================

    def __init__(self, name, age, salary):

        self.name = name          # PUBLIC
        self._age = age           # PROTECTED
        self.__salary = salary    # PRIVATE


    # ========================
    # PUBLIC FUNCTION
    # ========================

    def show_public(self):
        print("A Public:", self.name)
```

```python
    def _show_protected(self):
        print("A Protected:", self._age)


    # ========================
    # PRIVATE FUNCTION
    # ========================

    def __show_private(self):
        print("A Private:", self.__salary)


    # Private function ko access karne ke liye
    # public function
    def get_private(self):
        self.__show_private()
```

```python
class B(A):

    def __init__(self, name, age, salary):

        # A ka constructor call
        super().__init__(name, age, salary)

        # B ke apne attributes
        self.B_name = "B ka naam"
        self._B_age = 25
        self.__B_salary = 60000


    # ==========================
    # B PUBLIC FUNCTION
    # ==========================

    def B_public_function(self):
        print("B Public:", self.B_name)
```

```python
    def _B_protected_function(self):
        print("B Protected:", self._B_age)


    # ========================
    # B PRIVATE FUNCTION
    # ========================

    def __B_private_function(self):
        print("B Private:", self.__B_salary)


    def get_B_private(self):
        self.__B_private_function()


# ================================================
# OBJECT
# ================================================

s1 = B("Tarun", 22, 50000)
```

A ke members ko B ke object se access

Python

```python
# PUBLIC
print(s1.name)              # ✅
s1.show_public()            # ✅


# PROTECTED
print(s1._age)              # ⚠️ technically possible
s1._show_protected()        # ⚠️ technically possible


# PRIVATE
# print(s1.__salary)        # ❌
# s1.__show_private()       # ❌


# Private ko method ke through
s1.get_private()            # ✅
```

B ke members

Python

```python
# PUBLIC
print(s1.B_name)                    # ✅
s1.B_public_function()              # ✅


# PROTECTED
print(s1._B_age)                    # ⚠️
s1._B_protected_function()          # ⚠️


# PRIVATE
# print(s1.__B_salary)              # ❌
# s1.__B_private_function()         # ❌


# Private ko method ke through
s1.get_B_private()                  # ✅
```

```
s1 = B("Tarun", 22, 50000)
            ↓
       B.__init__()
            ↓
   super().__init__()
            ↓
       A.__init__()
            ↓
self.name       = "Tarun"     PUBLIC
self._age       = 22          PROTECTED
self.__salary   = 50000       PRIVATE
```

Upgr

`super()` ka main kaam hai **child class se parent class ke method/constructor ko call karna.**

Tumhare example se:

Python

Run

```python
class A:

    def __init__(self, name, age, salary):
        self.name = name
        self._age = age
        self.__salary = salary


class B(A):

    def __init__(self, name, age, salary):
        super().__init__(name, age, salary)
```

`this` kis language mein hota hai?

Language	Current object ko refer karne ke liye
Python	`self`
Java	`this`
JavaScript	`this`
C++	`this`
C#	`this`

Python

```python
class A:
    name = "tarun patidar"
    _age = 5
    __salary = 56.444

    def __init__(self, name, age, salary):
        self.name = name
        self._age = age
        self.__salary = salary

    def show(self, ispass=True):
        print("ispass =", ispass)


class B(A):
    _city = "nalkheda"
    budget = 555.44

    def __init__(self, name, age, salary):
        super().__init__(name, age, salary)
```

Python

```
    def show(self):
        print(super().name)
        print(super()._age)
        print(super()._A__salary)

        super().show(True)



b = B("Tarun Patidar", 5, 545.54)

b.show()
```

Flow

```
b = B("Tarun Patidar", 5, 545.54)
              ↓
      B.__init__()
              ↓
 super().__init__(...)
              ↓
      A.__init__()
              ↓
 self.name = "Tarun Patidar"
 self._age = 5
 self.__salary = 545.54
```

Phir:

```python
b.show()
```

Python

```
b.show()
```

B ka `show()` chalega:

Python

```
def show(self):
```

Uske andar:

Python

```
super().name
```

→ Parent `A` ka `name`

Python

```
super()._age
```

`super()` sirf constructor ke liye nahi hai

Parent ka koi bhi method call kar sakte ho:

‹/› Python

```python
class A:
    def show(self):
        print("A ka show")


class B(A):
    def show(self):
        super().show()
        print("B ka show")


s1 = B()
s1.show()
```

Output:

1. Single Inheritance

Ek parent → ek child.

Python

```python
class A:
    pass

class B(A):
    pass
```

```
A
↓
B
```

2. Multilevel Inheritance

Parent → Child → Grandchild.

Python

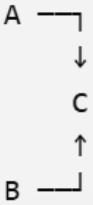
```python
class A:
    pass

class B(A):
    pass

class C(B):
    pass
```

3. Multiple Inheritance

Ek child ke **multiple parents**.

Python

```python
class A:
    pass

class B:
    pass

class C(A, B):
    pass
```

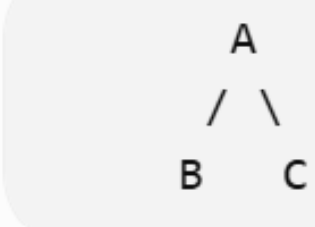


4. Hierarchical Inheritance

Ek parent → multiple children.

Python

```python
class A:
    pass

class B(A):
    pass

class C(A):
    pass
```



5. Hybrid Inheritance

Do ya zyada inheritance patterns ka combination.

Example:

Python

```python
class A:
    pass

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass
```


3. MRO kya hai?

MRO = Method Resolution Order

Simple meaning:

> Jab Python ko kisi method/attribute ko find karna ho, to **kis class me pehle search karega aur kis order me search karega**, us order ko MRO kehte hain.

Python

```python
class A:
    def show(self):
        print("A")


class B(A):
    pass


class C(B):
    pass
```

Agar:

Python

```python
obj = C()
obj.show()
```

Python search karega:

Python search karega:

```
C
↓
B
↓
A
```

`C` me nahi mila → `B` me nahi mila → `A` me mil gaya.

Isliye:

```
A
```

MRO dekhne ke liye:

```python
print(C.mro())
```

Output roughly:

```
[C, B, A, object]
```

4. Multiple inheritance me MRO bahut important hai

Python

```python
class A:
    def show(self):
        print("A")


class B(A):
    pass


class C(A):
    def show(self):
        print("C")


class D(B, C):
    pass
```

Python

```python
class A:

    def show(self):
        print("a")


class B(A):

    pass

    def show(self):
        print("B")


class C(A):

    pass

    def show(self):
        print("C")
```

python

```python
class D(B, C):
    pass


obj = D()
obj.show()       # B
A.show(obj)      # a
B.show(obj)      # B
C.show(obj)      # C
```

```
obj = D()
obj.show()
```

Python ko decide karna hai `show()` kahan se lena hai.

MRO:

```
Python

print(D.mro())
```

roughly:

```
D → B → C → A → object
```

`B` me `show()` nahi hai → `C` me mil gaya.

Output:

```
C
```

`super()`	MRO
Built-in function hai	**Method Resolution Order** hai
Current class ke baad MRO me aane wali class ki method ko access karta hai	Python classes ko **kis order me search karega**, ye batata hai
Code me `super()` likhte hain	`ClassName.mro()` se MRO dekh sakte hain
MRO ko follow karta hai	**Search order decide karta hai**
Example: `super().show()`	Example: `D → B → C → A → object`

Multiple inheritance me difference aur clear hota hai

Python

```python
class A:
    def show(self):
        print("A")


class B(A):
    def show(self):
        print("B")
        super().show()


class C(A):
    def show(self):
        print("C")
        super().show()


class D(B, C):
    def show(self):
        print("D")
```

Python

```python
class D(B, C):
    def show(self):
        print("D")
        super().show()


d = D()
d.show()
```

MRO:

Python

```python
print(D.mro())
```

roughly:

```
D → B → C → A → object
```

Aur `super()` isi MRO ko follow karta hai:

```
D → B → C → A → object
```

Aur `super()` isi MRO ko follow karta hai:

```
D.show()
   ↓
super()
   ↓
B.show()
   ↓
super()
   ↓
C.show()
   ↓
super()
   ↓
A.show()
```

2. B ke object se A ka `show()` bhi call karna

`super()` use karo:

Python

```python
class A:
    def show(self):
        print("A ka show")


class B(A):
    def show(self):
        print("B ka show")
        super().show()


b = B()

b.show()
```

Output:

```
B ka show
A ka show
```

Flow:

```
b.show()
    ↓
B.show()
    ↓
super().show()
    ↓
A.show()
```

1. Default arguments

Python

```python
class A:

    def add(self, a, b, c=0):
        print(a + b + c)


obj = A()

obj.add(10, 20)
obj.add(10, 20, 30)
```

Python

```python
class A:

    def add(self, *args):
        print(sum(args))


obj = A()

obj.add(10, 20)
obj.add(10, 20, 30)
obj.add(10, 20, 30, 40)
```

Output:

Python

```python
class A:

    def area(self, shape, *args):

        # Square
        if shape == "square":
            side = args[0]
            return side * side

        # Rectangle
        elif shape == "rectangle":
            length = args[0]
            breadth = args[1]
            return length * breadth

        # Circle
        elif shape == "circle":
            radius = args[0]
            return 3.14 * radius * radius
```

```python
        # Circle
        elif shape == "circle":
            radius = args[0]
            return 3.14 * radius * radius

        else:
            return "Invalid shape"


a = A()

print(a.area("square", 5))
print(a.area("rectangle", 5, 10))
print(a.area("circle", 5))
```

> define karna = operator overloading.

Simple example: `+`

Python

```python
class A:

    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return self.value + other.value


a1 = A(10)
a2 = A(20)

print(a1 + a2)
```

Haan bhai 👍 Python mein operator overloading ke liye **bahut saare magic/dunder methods** hote hain. Main important operators ko category-wise table mein de raha hoon.

1. Arithmetic Operators

Operator	Magic Method	Example
`+`	`__add__()`	`a + b`
`-`	`__sub__()`	`a - b`
`*`	`__mul__()`	`a * b`
`/`	`__truediv__()`	`a / b`
`//`	`__floordiv__()`	`a // b`
`%`	`__mod__()`	`a % b`
`**`	`__pow__()`	`a ** b`
`@`	`__matmul__()`	`a @ b`

2. Comparison Operators

Operator	Magic Method	Example
`==`	`__eq__()`	`a == b`
`!=`	`__ne__()`	`a != b`
`<`	`__lt__()`	`a < b`
`<=`	`__le__()`	`a <= b`
`>`	`__gt__()`	`a > b`
`>=`	`__ge__()`	`a >= b`

3. Bitwise Operators

Operator	Magic Method	Example
`&`	`__and__()`	`a & b`
`|`	`__or__()`	`a | b`
`^`	`__xor__()`	`a ^ b`
`~`	`__invert__()`	`~a`
`<<`	`__lshift__()`	`a << b`
`>>`	`__rshift__()`	`a >> b`

4. Assignment Operators

Compound assignment operators ke liye alag magic methods hote hain:

Operator	Magic Method
`+=`	`__iadd__()`
`-=`	`__isub__()`
`*=`	`__imul__()`
`/=`	`__itruediv__()`
`//=`	`__ifloordiv__()`
`%=`	`__imod__()`
`**=`	`__ipow__()`
`&=`	`__iand__()`

`&=`	`__iand__()`
`|=`	`__ior__()`
`^=`	`__ixor__()`
`<<=`	`__ilshift__()`
`>>=`	`__irshift__()`

5. Unary Operators

Operator	Magic Method	Example
`+a`	`__pos__()`	`+a`
`-a`	`__neg__()`	`-a`
`~a`	`__invert__()`	`~a`
`abs(a)`	`__abs__()`	`abs(a)`

6. `in` Operator

Operator	Magic Method	Example
`in`	`__contains__()`	`x in obj`
`not in`	`__contains__()`	`x not in obj`

Python

```python
class A:
    def __contains__(self, item):
        return item == 10

obj = A()

print(10 in obj)       # True
print(20 in obj)       # False
```

Python

```python
class Number:

    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return Number(self.value + other.value)

    def __sub__(self, other):
        return Number(self.value - other.value)

    def __mul__(self, other):
        return Number(self.value * other.value)

    def __mod__(self, other):
        return Number(self.value % other.value)

    def __str__(self):
        return str(self.value)
```

```python
a = Number(10)
b = Number(3)

print(a + b)    # 13
print(a - b)    # 7
print(a * b)    # 30
print(a % b)    # 1
```

Internally kya hota hai?

Jab tum:

```python
a1 + a2
```

likhte ho, Python roughly:

```python
a1.__add__(a2)
```

call karta hai.

```
a1 + a2
  ↓
a1.__add__(a2)
  ↓
30
```

`gt` = greater than

```python
class A:
    def __init__(self, value):
        self.value = value

    def __gt__(self, other):
        return self.value > other.value


a = A(20)
b = A(10)

print(a > b)
```

Output:

```
True
```

1. Class Attribute

Jo variable **class ke andar directly** banaya jata hai, wo class attribute hota hai.

Python

```python
class Student:
    college = "Medi-Caps"
    course = "CSE"
```

`college` aur `course` **class attributes** hain.

Inhe class ke naam se access kar sakte ho:

Python

```python
print(Student.college)
print(Student.course)
```

2. Instance Attribute

Jo variable `__init__()` ke andar `self` **ke saath** banaya jata hai, wo instance attribute hota hai.

Python

```python
class Student:

    college = "Medi-Caps"       # Class attribute

    def __init__(self, name, age):
        self.name = name        # Instance attribute
        self.age = age          # Instance attribute
```

Objects:

Python

```python
s1 = Student("Tarun", 22)
s2 = Student("Karna", 21)
```

Ab:

Method + Instance Attribute

Python

```python
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def show(self):
        print(self.name)
        print(self.age)


s1 = Student("Tarun", 22)

s1.show()
```

Output:

```
Tarun
22
```

1. Instance Method

`self` use karta hai.

Python

```python
class Student:

    def show(self):
        print("Student")


s1 = Student()
s1.show()
```

2. Class Method

`cls` use karta hai aur `@classmethod` lagta hai.

Python

```python
class Student:

    college = "Medi-Caps"

    @classmethod
    def show_college(cls):
        print(cls.college)


Student.show_college()
```

1. Static method kya hota hai?

Upgrad

Class ke andar aisa method jise **object (`self`) ya class (`cls`) ke data ki zarurat nahi hoti**, use static method bana sakte hain.

Python me:

Python Run

```
@staticmethod
```

lagate hain.

Python

```
class Calculator:

    @staticmethod
    def add(a, b):
        return a + b


print(Calculator.add(10, 20))
```

Output:

```
30
```

Yahan `self` nahi hai:

Python

```
def add(a, b):
```

kyunki method ko kisi object ke data ki zaru nahi hai.

2. Normal method vs Static method

Instance method

```python
class Calculator:

    def add(self, a, b):
        return a + b


c = Calculator()

print(c.add(10, 20))
```

Yahan Python automatically `c` ko `self` me bhejta hai:

```
c.add(10, 20)
 ↓
Calculator.add(c, 10, 20)
```

Static method

Python

```python
class Calculator:

    @staticmethod
    def add(a, b):
        return a + b


print(Calculator.add(10, 20))
```

Yahan koi `self` automatically nahi jaata:

```
Calculator.add(10, 20)
 ↓
add(10, 20)
```

5. Static method ka real example

Maan lo Student ke marks check karne hain:

Python

```python
class Student:

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    @staticmethod
    def is_pass(marks):
        return marks >= 40


s1 = Student("Tarun", 75)

print(Student.is_pass(75))
```

3-1f21c39adb71

6. Instance method, Class method, Static method

Python

```python
class Student:

    college = "Medi-Caps"

    def __init__(self, name):
        self.name = name

    # Instance method
    def show_name(self):
        print(self.name)

    # Class method
    @classmethod
    def show_college(cls):
        print(cls.college)

    # Static method
    @staticmethod
    def add(a, b):
        print(a + b)
```

Use:

```python
s1 = Student("Tarun")

s1.show_name()

Student.show_college()

Student.add(10, 20)
```

Output:

```
Tarun
Medi-Caps
30
```

`@property` decorator — Full Definition

Up

> Python ka `@property` decorator kisi class ke method ko attribute ki tarah access karne ki facility deta hai, yani method ko call karte waqt `()` lagane ki zarurat nahi hoti. Iska use mainly encapsulation ke liye kiya jata hai, jisse hum kisi attribute ko read karne, validate karne ya modify karne ka control rakh sakte hain.

Simple example

Python

Run

```python
class Student:

    def __init__(self, age):
        self._age = age

    @property
    def age(self):
        return self._age


s = Student(22)
```

```
print(s.age)
```

Yahan:

```
Python

s.age
```

likha hai, `s.age()` **nahi**.

Lekin internally `age()` method hi execute ho raha hai.

```
s.age
 ↓
@property
 ↓
age() method
 ↓
self._age
 ↓
22
```

2. `@property` ki zarurat kyu?

Maan lo:

```python
class Student:

    def __init__(self, age):
        self.age = age
```

User directly:

```python
s.age = -100
```

kar sakta hai.

Agar validation chahiye:

Agar validation chahiye:

Python

```python
class Student:

    def __init__(self, age):
        self._age = age

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):

        if value < 0:
            print("Age invalid")
        else:
            self._age = value
```

Ab:

```python
s = Student(22)

print(s.age)

s.age = 25
print(s.age)

s.age = -10
```

Output:

```
22
25
Age invalid
```

Yahan `age` ko access karne par:

4. Setter

Setter ka matlab:

> **Value set/change karna.**

‹/› Python

```python
@age.setter
def age(self, value):
    self._age = value
```

Use:

‹/› Python

```python
s.age = 25
```

5. Getter + Setter complete example

Python

```python
class Student:

    def __init__(self, age):
        self._age = age

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):

        if value >= 0:
            self._age = value
        else:
            print("Age cannot be negative")
```

Python

```python
s = Student(22)

print(s.age)

s.age = 25
print(s.age)

s.age = -5
print(s.age)
```

Output:

```
22
25
Age cannot be negative
25
```

Bilkul bhai. **Dunder function/method** ki proper definition:

Dunder Method — Full Definition

> **Python me Dunder Method (Double Underscore Method) wo special method hota hai jiske naam ke beginning aur end dono taraf double underscore (`__`) hota hai. Ye methods Python ke built-in operations aur language behavior ko define ya customize karne ke liye use hote hain. Inhe Python automatically particular situations me call karta hai, ya hum explicitly bhi call kar sakte hain.**

Dunder = Double Underscore

Example:

Python

Run

```
__init__()
__str__()
__add__()
__eq__()
```

Example: `__add__()`

Operator overloading me:

Python

```python
class A:

    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return self.value + other.value


a = A(10)
b = A(20)

print(a + b)
```

b3-1f21c39adb71

1. Diamond Problem kya hai?

Structure dekho:

```
   A
  / \
 B   C
  \ /
   D
```

Yahan:

- `B`, `A` se inherit karta hai
- `C`, `A` se inherit karta hai
- `D`, `B` aur `C` dono se inherit karta hai

```python
class A:
    def show(self):
        print("A")


class B(A):
    pass


class C(A):
    pass


class D(B, C):
    pass
```

Ab:

2. Python kaise solve karta hai?

Python **MRO (Method Resolution Order)** use karta hai.

```python
print(D.mro())
```

Output roughly:

```
[D, B, C, A, object]
```

Python isi order me method search karega:

```
D
 ↓
B
 ↓
C
 ↓
A
 ↓
object
```

3. `super()` ke saath Diamond Problem

Ab important example:

Python

```python
class A:

    def show(self):
        print("A")


class B(A):

    def show(self):
        print("B")
        super().show()


class C(A):

    def show(self):
        print("C")
        super().show()
```

```
class D(B, C):

    def show(self):
        print("D")
        super().show()


d = D()

d.show()
```

Output:

```
D
B
C
A
```

Python me Abstraction

Python me commonly `ABC` **(Abstract Base Class)** aur `@abstractmethod` use karke abstraction implement karte hain.

Python

```python
from abc import ABC, abstractmethod


class Vehicle(ABC):

    @abstractmethod
    def start(self):
        pass
```

Yahan `Vehicle` ek **abstract class** hai.

`start()` ek **abstract method** hai.

Ab child class ko `start()` implement karna padega:

Ab child class ko `start()` implement karna padega:

Python

```python
class Car(Vehicle):

    def start(self):
        print("Car starts with key")


class Bike(Vehicle):

    def start(self):
        print("Bike starts with self start")
```

Use:

Python

```python
car = Car()
bike = Bike()

car.start()
bike.start()
```

Python

```python
from abc import ABC, abstractmethod


class Payment(ABC):

    @abstractmethod
    def pay(self, amount):
        pass


class UPI(Payment):

    def pay(self, amount):
        print("UPI se ₹", amount, "paid")


class Card(Payment):

    def pay(self, amount):
        print("Card se ₹", amount, "paid")
```

```
p1 = UPI()
p2 = Card()

p1.pay(500)
p2.pay(1000)
```

Output:

```
UPI se ₹ 500 paid
Card se ₹ 1000 paid
```

Copy Constructor kya hota hai?

> **Copy constructor ek aisa constructor/method hota hai jo kisi existing object ki values ko use karke ek naya object create karta hai.**

Simple:

```
Existing object
      ↓
   copy data
      ↓
New object
```

-1f21c39adb71

Python

```python
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def copy_student(cls, other):
        return cls(other.name, other.age)


s1 = Student("Tarun", 22)

s2 = Student.copy_student(s1)

print(s1.name, s1.age)
print(s2.name, s2.age)
```

Output:

```
Tarun 22
Tarun 22
```

Ye teen alag concepts hain:

Concept	Meaning
Copy Constructor	Existing object se naya object banana
Shallow Copy	Outer object copy, nested objects same reference
Deep Copy	Outer + nested objects sabki separate copy

Example:

Python

Run

```python
import copy

s2 = copy.copy(s1)       # Shallow copy
s3 = copy.deepcopy(s1)   # Deep copy
```

Python

```python
import copy

class Student:

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks


s1 = Student("Tarun", [80, 90, 70])

# Deep copy
s2 = copy.deepcopy(s1)

print(s1.name)
print(s1.marks)

print(s2.name)
print(s2.marks)
```

2. Object ko manually delete karna

Python me `del` se object ka reference remove kar sakte ho:

Python

```python
class Student:

    def __del__(self):
        print("Destructor called")


s1 = Student()

del s1

print("Done")
```

Output:

```
Tarun
[80, 90, 70]
Tarun
[80, 90, 70]
```

Ab important part:

Python

```python
s2.marks[0] = 100

print(s1.marks)
print(s2.marks)
```

Output:

```
[80, 90, 70]
[100, 90, 70]
```

Original `s1` ki list change nahi hui. ✅

Destructor kya hota hai?

> Destructor ek special/dunder method hai jo object destroy hone ke samay Python dwara call kiya ja sakta hai. Python me destructor ka method `__del__()` hota hai.

Python

Run

```python
class Student:

    def __del__(self):
        print("Object destroy ho raha hai")
```

e

1. `dir()` kya hai?

`dir()` kisi **object/class/module ke available attributes aur methods** ki list deta hai.

Python

Run

```python
class Student:
    name = "Tarun"

    def show(self):
        print("Hello")

s = Student()

print(dir(s))
```

Output mein bahut saare names aayenge, jaise:

```
['__class__', '__dict__', '__module__', 'name', 'show', ...]
```

👉 `dir()` = "Is object ke paas kya-kya available hai?"

2. `__dict__` kya hai?

`__dict__` object/class ke **attributes ko dictionary ke form mein** dikhata hai.

Python

```python
class Student:
    name = "Tarun"

    def __init__(self, age):
        self.age = age

    def show(self):
        print("Hello")

s = Student(21)

print(s.__dict__)
```

Output:

```
{'age': 21}
```

Yahan `age` instance ke andar stored hai.

`dir()` vs `__dict__`

	`dir()`	`__dict__`
Purpose	Available names dikhana	Stored attributes dikhana
Output	List	Dictionary
Methods dikhata hai?	✅ Yes	Class ke case mein ✅
Object attributes	✅	✅
Inherited members	✅ Generally list mein	❌ Directly nahi
Example	`dir(s)`	`s.__dict__`

🔥 Python Introspection Methods

Function / Method	Kaam	Example
`dir()`	Object ke available attributes/methods ki list	`dir(obj)`
`__dict__`	Object/class ke attributes dictionary mein	`obj.__dict__`
`type()`	Object ka type batata hai	`type(obj)`
`id()`	Object ki identity/ID batata hai	`id(obj)`
`isinstance()`	Check karta hai object kisi class ka instance hai ya nahi	`isinstance(obj, Student)`
`issubclass()`	Check karta hai ek class doosri class ki subclass hai ya nahi	`issubclass(B, A)`
`hasattr()`	Check karta hai attribute exist karta hai ya nahi	`hasattr(obj, "name")`

`issubclass()`	Check karta hai ek class doosri class ki subclass hai ya nahi	`issubclass(B, A)`
`hasattr()`	Check karta hai attribute exist karta hai ya nahi	`hasattr(obj, "name")`
`getattr()`	Attribute ki value dynamically nikalta hai	`getattr(obj, "name")`
`setattr()`	Attribute dynamically create/change karta hai	`setattr(obj, "age", 21)`
`delattr()`	Attribute dynamically delete karta hai	`delattr(obj, "age")`
`callable()`	Check karta hai object ko function ki tarah call kar sakte hain ya nahi	`callable(obj)`

Ek hi code mein 👇

Python

```python
class Student:
    name = "Tarun"

    def show(self):
        print("Hello")


class B(Student):
    pass


s = B()

print(type(s))                  # <class '__main__.B'>

print(id(s))                    # Object ki identity

print(dir(s))                   # Available attributes/methods

print(s.__dict__)               # Instance ke stored attributes
```

python

```python
print(dir(s))                          # Available attributes/methods

print(s.__dict__)                      # Instance ke stored attributes

print(isinstance(s, Student))          # True
print(isinstance(s, B))                # True

print(issubclass(B, Student))           # True

print(hasattr(s, "name"))              # True

print(getattr(s, "name"))              # Tarun

setattr(s, "age", 21)
print(s.age)                           # 21

delattr(s, "age")
print(hasattr(s, "age"))               # False

print(callable(s.show))                # True
```